

408 真题错题本

2018 年

选择题

3 W A

在 408 考试中，当使用具体的编程语言（如 C）实现算法时，默认数组下标从 0 开始；但在算法的描述、推导和伪代码中，为了方便，常常使用从 1 开始的下标。考生需要具备在这两种模式间灵活转换的能力

8 W A

m 阶 B 树，最小度 $t = \lceil m/2 \rceil$

- 1. 根节点关键字数：闭区间 $[1, m - 1]$
- 2. 非根节点关键字数：闭区间 $[t - 1, m - 1]$
- 3. 节点孩子数 = 节点关键字数 + 1
 - 根节点若 $h = 0$ 则孩子数为 0
 - 非根节点若为叶结点则孩子数为 0

利用树形根据等比数列求和公式推导即可

扩展

直观理解：每个非根节点至少填充了一半，这样可以保证树整体高度最小化，提高查找效率

11 W

上滤：与父节点比较，若破坏堆序性就与父节点交换

下滤：与两个子节点比较，若为大根堆，则与较大的子节点交换，若为小根堆则与较小的子节点交换

建堆方法

- 1. 自顶向下建堆法：将元素插入堆，上滤，适用于优先队列 $O(n \log n)$
- 2. 自下而上建堆法：倒序对每个非叶节点进行下滤，适用于堆化 $O(n)$

13 A

1	FFFFFDF
2	+ 21
3	-----
4	10000000

$$x = (-21)_{16} = (-33)_{10}$$

$$y = (41)_{16} = (65)_{10}$$

法一：直接减

1	FFFFFDF
2	- 41
3	-----
4	FFFFF9E

法二：转换为相反数然后加

1	-0000041H = FFFFFBF
2	FFFFFDF
3	+ FFFFFBF
4	-----
5	1FFFFF9E

14 W A

若阶码 **不全为 0** 也 **不全为 1**，则为规格化浮点数

最小的正规格化数 $0/00000001/00\cdots00$ ，即 1.0×2^{-126}

17 W A

DRAM 采用的是行列地址线复用技术，我们尽量选用行列差值不要太大的

DRAM 一般是按行刷新的，所以应选行数值较少的。类比二维数组，也是行数少的性能较高

DRAM 芯片行、列数的优化原则：

假定有一个 $2^n \times b$ 位 DRAM 芯片的存储阵列，其行数为 r ，列数为 c ，则 $2^n = rc$ 。存储阵列的地址位数为 n ，其中行地址位数为 $\log_2 r$ ，列地址位数为 $\log_2 c$ ，则 $n = \log_2 r + \log_2 c$ 。由于 DRAM 芯片采用地址引脚复用技术，为减少地址引脚数，应尽量使行、列位数相同，即满足 $|r - c|$ 最小。又由于 DRAM 按行刷新，为减少刷新开销，应使行数较少，因此还需满足 $r \leq c$

扩展

DRAM 芯片行列地址线复用时，为什么选择行数较少的芯片比行数较多的芯片性能更高？

- DRAM 芯片采用 **行列地址复用** 技术，一条地址线既用于行地址也用于列地址
- 行数较少意味着每行的存储单元更多，因此 **访问同一行的数据可以一次性读出更多信息**，减少行切换次数
- DRAM 按行刷新和按行访问，如果行数多，每行切换和刷新次数增多，会增加延迟，降低性能
- 因此在相同行容量下，**选择行数较少、列数较多** 的 DRAM 可以获得更高的存取性能

类比二维数组：行少、列多 $\rightarrow$ 连续访问更高效

20 A

指令流水线的每个流水段的时间单位为时钟周期，题中指令流水线的指令需要用到 $A \sim E$ 五个部件，所以每个流水段的时间应取：**最大部件时间** $80ps$ ，此外还有 **寄存器延时** 为 $20ps$ ，则 CPU 的时钟周期为 $100ps$

流水段寄存器

每个流水段后面都有一个流水段寄存器，用于锁存本段处理完的所有数据，以保证本段的执行结果能在下个时钟周期给下一流水段使用

在考试中，若没有明确说明，则可以不用考虑流水寄存器的时延

扩展

CPU 时钟周期的倒数就是 CPU 主频

21 W

总线数据传输率 = 总线工作频率 $\times$ 总线宽度 / 8, 所以 I 和 II 会影响总线数据传输率

采用 **突发传输方式** (也称猝发传输)，在一个总线周期内传输存储地址连续的多个数据字，从而提高了传输效率

采用地址/数据线复用只是减少了线的数量，节省了**成本**，并不能提高传输率

22 A

中断隐指令 完成的工作有

1. 关中断
2. 保存断点
3. 引出中断服务程序

通用寄存器的保护由 **中断服务程序** 完成

中断允许状态即开中断后，才能响应中断请求

24 A

例题 [2018 真题]: 某系统采用基于优先权的非抢占式进程调度策略, 完成一次进程调度和进程切换的系统时间开销为 $1\mu s$ 。在 T 时刻就绪队列中有 3 个进程 P_1 、 P_2 和 P_3 , 其参数如下:

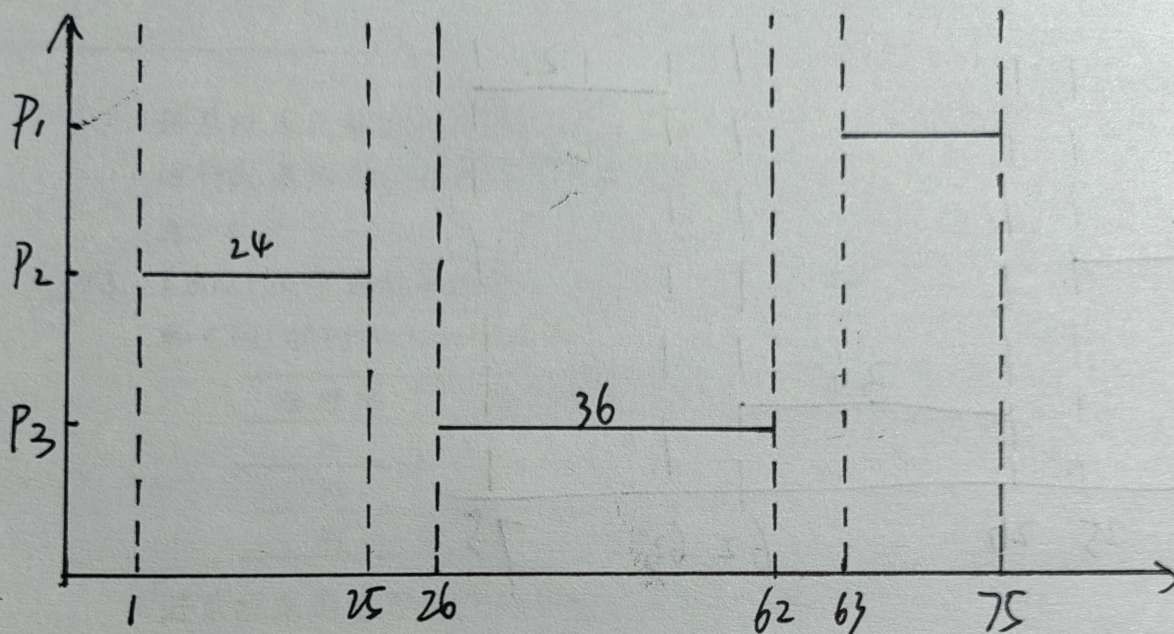
进程名	等待时间	需要的 CPU 时间	优先权
P_1	$30\mu s$	$12\mu s$	10
P_2	$15\mu s$	$24\mu s$	30
P_3	$18\mu s$	$36\mu s$	20

若优先权值大的进程优先获得 CPU (优先权 $30 > 20 > 10$) , 从 T 时刻起系统开始进程调度, 则系统的平均周转时间为 () 。
A. $54\mu s$ B. $73\mu s$ C. $74\mu s$ D. $75\mu s$

解答

这个题坑就坑在到达时间需要推导 (这里是重点, 以后遇到等待时间要注意)

- P_1 到达时间 = $T - 30\mu s$
- P_2 到达时间 = $T - 15\mu s$
- P_3 到达时间 = $T - 18\mu s$



优先级 $P_2 > P_3 > P_1$

周转时间：

$$P_1 \quad (75 - 0) + 30 = 105 \mu s$$

$$P_2 \quad (25 - 0) + 15 = 40 \mu s$$

$$P_3 \quad (62 - 0) + 18 = 80 \mu s$$

$$\text{平均周转时间} = \frac{105 + 40 + 80}{3} = 75 \mu s$$

故选 D

27 W

进程等待某资源为可用（不包括处理机）或等待 IO 完成均会进入 **阻塞状态**

系统将 CPU 分给高优先权的进程，进程进入就绪状态

28 W

“条件变量”是管程内部说明和使用的一种特殊变量，其作用类似于信号量机制中的“信号量”，都是用于实现进程同步的。

需要注意的是，在同一时刻，管程中只能有一个进程在执行。

如果进程 A 执行了 `x.wait()` 操作，那么该进程会阻塞，并挂到条件变量 x 对应的阻塞队列上。

这样，管程的使用权被释放，就可以有另一个进程进入管程。

如果进程 B 执行了 `x.signal()` 操作，那么会唤醒 x 对应的阻塞队列队头进程。

在 Pascal 语言的管程中，规定只有一个进程要离开管程时才能调用 `signal()` 操作。

管程中 `x.wait()` 与信号量中的 `wait()` 区别：管程的 `wait()` 是关于 **条件等待** 的，会自动释放锁；信号量的 `wait()` 是关于 **资源获取** 的，不涉及锁的自动管理

29 W A

时钟中断的主要工作是处理和时间有关的信息以及决定是否执行调度程序，**和时间有关的所有信息**，包括系统时间、进程的时间片、延时、使用 CPU 的时间、各种定时器，故 I、II、III 均正确，选 D。

其他冷门中断类型

中断类型	主要功能	触发条件/说明
机器校验中断	处理硬件故障，如电源故障、主存校验错误，确保系统在严重硬件错误时能安全处理	由设备故障引发。通常为 不可屏蔽中断 ，优先级最高
访管中断	用户程序 请求操作系统服务 （如 I/O 操作、创建进程）的桥梁，实现用户态向内核态的切换	由执行 访管指令 （如 x86 的 INT 指令）主动引发
程序性中断（异常）	处理程序运行时自身错误或异常，如除零、地址越界、非法指令、断点调试	由 当前正在执行的程序 本身的问题引发，属于 不可屏蔽中断
外部中断（狭义）	响应来自 CPU 外部的异步事件，如 定时器中断 、按键中断	由外部信号触发，与当前执行的指令无关
I/O 中断	由 I/O 控制器产生，用于通知 CPU 某个 I/O 操作已经完成或就绪	I/O 操作完成时触发

关于访管指令和自陷指令，相同点：二者都会引起处理器从用户态切换到内核态

特征	访管指令	自陷指令
触发方式	程序主动执行指令	硬件自动检测异常条件
预期性	预期和计划内的	通常是非预期的
同步性	同步	可能是同步或异步
控制权转移	程序控制流的一部分	异常控制流
目的	请求操作系统服务	处理异常情况
例子	系统调用、I/O 请求	除零、页故障、断点

32 W

硬件方法（swap 指令、TestAndSet 指令）实现进程同步不能实现让权等待

Peterson 算法满足有限等待但不满足让权等待

记录型信号量由于引入阻塞机制，消除了不让权等待的情况

让权等待：当进程不能进入自己的临界区时，应立即释放处理机，以免进程陷入“忙等”状态（受惠的是其他进程）

35 A

IEEE 802.11 无线局域网的 MAC 协议 CSMA/CA 进行信道预约的方法是交换 RTS 与 CTS 帧

扩展

CSMA/CA 的可选协议 RTS/CTS 协议

- RTS: Ready to send
- CTS: Clear to send
- 计算机 A 想给 AP 发送信息，首先发送 RTS 帧，AP 收到后会通知局域网内其他主机停止发送，在 AP 确认信道清空后，给 A 回复 CTS 帧，A 开始发送数据

数据帧格式（地址 1、地址 2、地址 3）： 去往 AP 中止，来自 AP 止中起

36 W S

变量说明

- n : 帧序号比特数
- k : 发送窗口
- t_1 : 分组发送时延 (传输时延)
- t_2 : 确认帧发送时延 (传输时延)
- τ : 单向传播时延
- x : 数据帧长度
- v : 数据传输速率

$$\text{信道利用率 } U = \frac{kt_1}{t_1 + 2\tau + t_2}$$

注意这里是 2τ , 也即 RTT

二刷

$$t_1 = \frac{x}{v}$$

- **停止等待 S-W**: $k = 1$, 简单可靠但效率低
- **回退 N 帧 GBN**: $k = 2^n - 1$, 平衡效率与复杂度
- **选择重传 SR**: $k = 2^{n-1}$, 效率最高但实现复杂

39 W A M

传输层分用的定义是: 接收方的传输层剥去报文首部后, 将数据分开并交付给目的进程。C 和 D 选项显然不符。

端口号是传输层服务访问点 TSAP, 用来标识主机中的应用进程。

对于 A 和 B 选项, **源端口号是在需要对方回信时选用**, 不需要时可用全 0。

目的端口号是在终点交付报文时使用到, 符合题意, 故选 B。

二刷

分用这个名称如何理解: 操作系统利用目的端口号作为标识, 将入站数据报正确地路由到等待它的应用进程套接字中 (分配使用)

注意与数据链路层的分片区分

大题

41

```
1  /**
2   * @see P20 T13
3   *
4   * 使用哈希对数组计数, 找到数组中的最大元素m, 从1枚举到m, 若计数为0则找到了未出现的最小正整数, 若不存在返回m+1
5   * 不允许使用STL
6   *
7   * 时间复杂度 O(n)
8   * 空间复杂度 O(n)
9   */
10 int findMinNotExists(int data[], int length) {
11     int max_ = 0;
12     for(int i = 0; i < length; ++i) {
13         if(data[i] > 0) {
14             max_ = std::max(max_, data[i]);
15         }
16     }
17     int* hash = (int*) malloc(sizeof(int) * (max_ + 1));
18     for(int i = 0; i <= max_; ++i) {
19         hash[i] = 0;
20     }
21     for(int i = 0; i < length; ++i) {
22         if(data[i] > 0) {
```

```

23     hash[data[i]]++;
24 }
25 }
26 int ans = -1;
27 for(int i = 1; i <= max_; ++i) {
28     if(hash[i] == 0) {
29         ans = i;
30         break;
31     }
32 }
33 free(hash);
34 return ans == -1 ? max_ + 1 : ans;
35 }

```

43

定时查询 IO，最长轮询间隔为填满数据缓冲区的时间

$$\text{IO 占 CPU 总时间的百分比} = \frac{1s \text{用于IO的时钟周期数}}{\text{CPU主频即1s的时钟周期数}} = \frac{1s \text{IO次数} \times \text{每次时钟周期数}}{\text{CPU主频}}$$

若 **准备数据的时间** 小于 **中断响应和中断处理的时间**，那么数据就会被刷新，造成丢失

$$1s \text{ 内 IO 次数} = \frac{1s}{\text{每次IO用时}}$$

二刷

实际上对于求 IO 占 CPU 总时间的百分比的题， $\frac{1s \text{用于IO的时钟周期数}}{\text{CPU主频}}$ 这个公式很有用

- 对于 **定时查询 IO 方式**（轮询）： $1s \text{用于IO的时钟周期数} = \frac{\text{数据传输率} \times 1s}{\text{数据缓冲寄存器大小}} \times$
 - 最大查询间隔 = $\frac{\text{数据缓冲寄存器大小}}{\text{数据传输率}}$ ，也就是每隔这么多时间必须查询一次，否则缓冲区会溢出
 - $1s$ 内的查询次数 = $\frac{\text{数据传输率} \times 1s}{\text{数据缓冲寄存器大小}}$ ，将这个次数乘以每次 IO 所需指令条数再乘以 CPI 即得 $1s \text{用于IO的时钟周期数}$
- 对于中断 IO 方式，无非是将定时查询 IO 方式的每次 IO 所需指令条数换成中断响应和中断处理的指令条数或时钟周期数
- 对于 DMA 方式，无非是将数据缓冲寄存器换成每次 DMA 传送的数据块
 - 注意：DMA 方式下，CPU 虽无需干预每个数据的传输，但必须为每一个数据块的传输进行 **预处理** 和 **后处理**

44 A

第二问：物理地址分为三段，只可能是两种情况：直接映射和组相联映射。这里中间那一段引出的线分别连接了两个 Cache 行，可以推断是 2 路组相联

Cache 的总容量：由物理地址结构知，Cache 共 8 组（看组号位数），每组 2 行（2 路），每行数据部分为 $2^5 = 32B$ （看偏移量位数）。此外还需要考虑有效位 1 位、替换算法位 1 位、脏位 1 位，标记 20 位，故总容量为

$$8 \times 2 \times (20 + 1 + 1 + 1 + 2^5 \times 8) = 4464b = 558B$$

第四问最后一空：这个虚拟地址无法转换为物理地址，但是不影响算组号，因为低 12 位是不变的，故组号为 3

二刷

如何看出全相联映射：TLB 每一行都有一个比较器，说明虚页号依次比较每个 TLB 行，说明是全相联映射（可以把页表内容调入任一块空 TLB 项中）

SRAM 与 DRAM 区别

特点	SRAM	DRAM
存储信息	双稳态触发器	电容
破坏性读出	否	是
需要刷新	不要	需要
送行列地址	同时送	分两次送（复用）

特点	SRAM	DRAM
运行速度	快	慢
集成度	低	高
存储成本	高	低
主要用途	高速缓存	主机内存

45

CLOCK 置换算法：访问位

改进型 CLOCK 置换算法：访问位、**修改位**（推迟页面写回磁盘的时间，减少 IO 次数）

46

纯粗心

是簇存地址项，而不是文件索引节点存地址项，故一个簇可以存 $\frac{4KB}{4B} = 1024$ 个地址项

于是整道题计算的都是错误的

注：文件索引节点 `inode` 存储文件元数据（metadata）

```
1 struct ext4_inode {
2     __le16 i_mode;           // 文件模式和类型
3     __le16 i_uid;           // 所有者UID
4     __le32 i_size_lo;       // 文件大小
5     __le32 i_atime;         // 访问时间
6     __le32 i_ctime;         // 创建时间
7     __le32 i_mtime;         // 修改时间
8     __le32 i_blocks_lo;     // 块计数
9     __le32 i_block[15];     // 数据块指针（12直接+3间接）
10    // ... 其他字段
11};
```

47

第一问：算这个子网下有多少个 IP，可以用 $254 - 129 + 1 = 126$ ，也可以用 $2^7 - 2 = 126$ 。已经分配了 $208 - 129 + 1 = 80$ 台，还可以分配 $126 - 80 - 1 = 45$ 台

为什么要再减一：在子网中，网关或路由器的接口通常需要一个 IP 地址，这个地址虽然属于可用主机地址范围，但不能分配给普通主机

第二问（纯粗心）：IP 分片需要保证数据部分长度为 8 的倍数，片偏移为下标除以 8（以 8 为单位）

2019 年

选择题

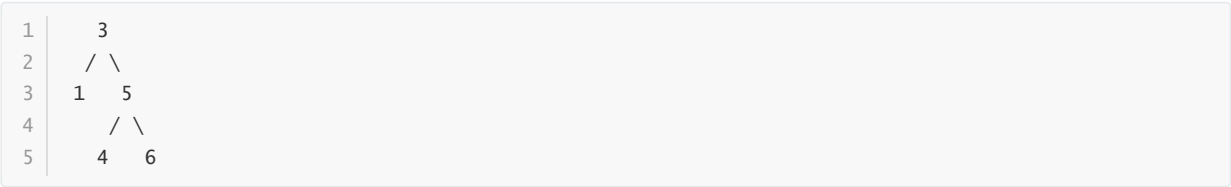
3 A

解方程得答案

$n = n_0 + n_2, n_0 = n_2 + 1$

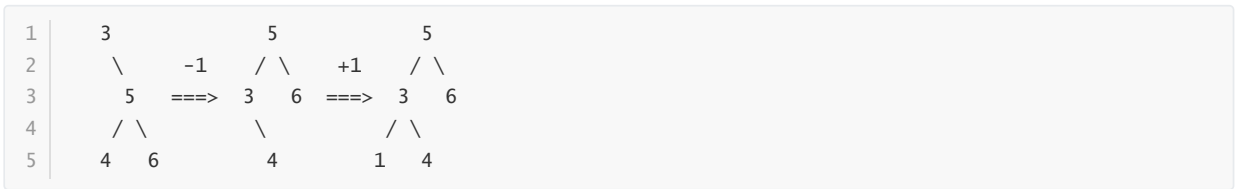
4 W A S

一个很好的例子，用来推导 AVL 树的题（其他题也可以用，很经典的例子）



删掉叶结点 4 或 6，树仍然平衡，插入后与之前相同

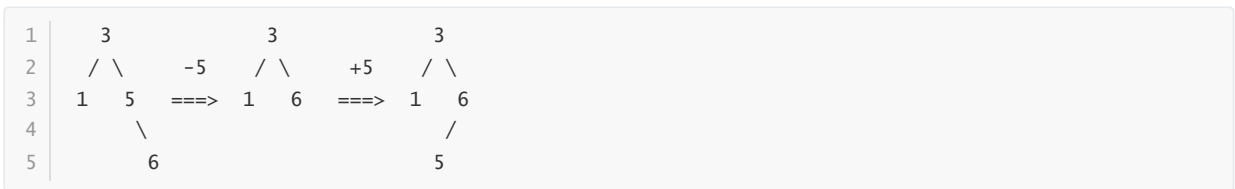
删掉叶结点 1，显然与之前不同，于是 I 正确



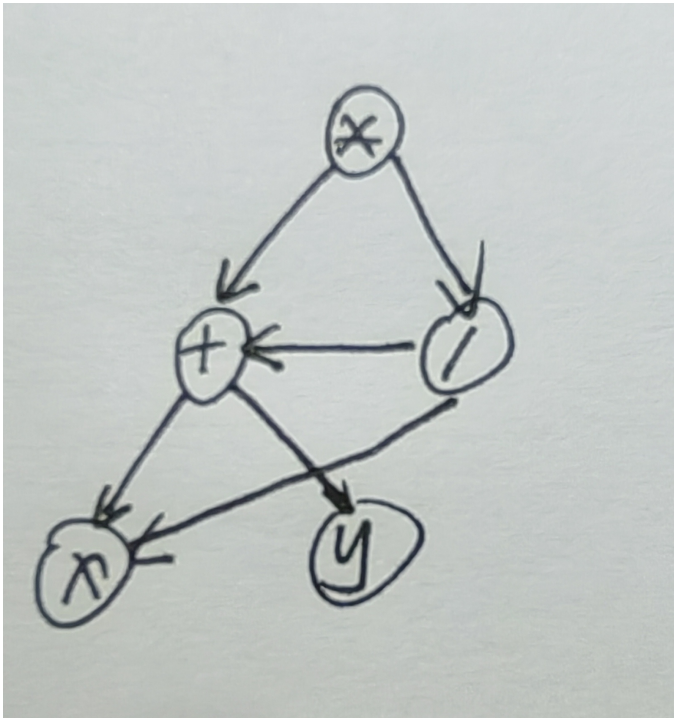
对于 II 和 III，删掉非叶结点 5，叶结点 4 上去代替 5，插入后与之前相同



稍微改一下初始的树，若为以下这种情况，删除 5 后，6 来替代，再插入 5 显然变化了



6 A



使用有向无环图表示表达式，规则如下

1. 节点类型：
 - 叶子节点：代表操作数（如变量 x 和 y ）
 - 内部节点：代表运算符（如 $+$, $\times$, $/$ ），每个内部节点有指向其操作数的边
2. 共享子表达式：如果同一个子表达式在表达式中多次出现，则只创建一个节点，并被多个父节点共享。这确保了图是无环的（即没有循环依赖）
3. 构建顺序：从最内层的子表达式开始构建，逐步向外，并使用哈希表或其他机制记录已创建的节点，以便重用

7 A

数据的初始状态：数据原始有序和无序对于不同排序算法影响很大

8 A

- 注意：查询失败的 ASL 的分母是散列值个数！
 - $ASL_{success} = \text{成功总比较次数} / \text{关键字个数}$
 - $ASL_{fail} = \text{失败总比较次数} / \text{散列值个数}$
- 同类型题：2023 年第 9 题，但是本题没有散列值个数这个坑

9 A

KMP：408 选择题中模式串 next 数组的求法：（408 要的是 0-based）

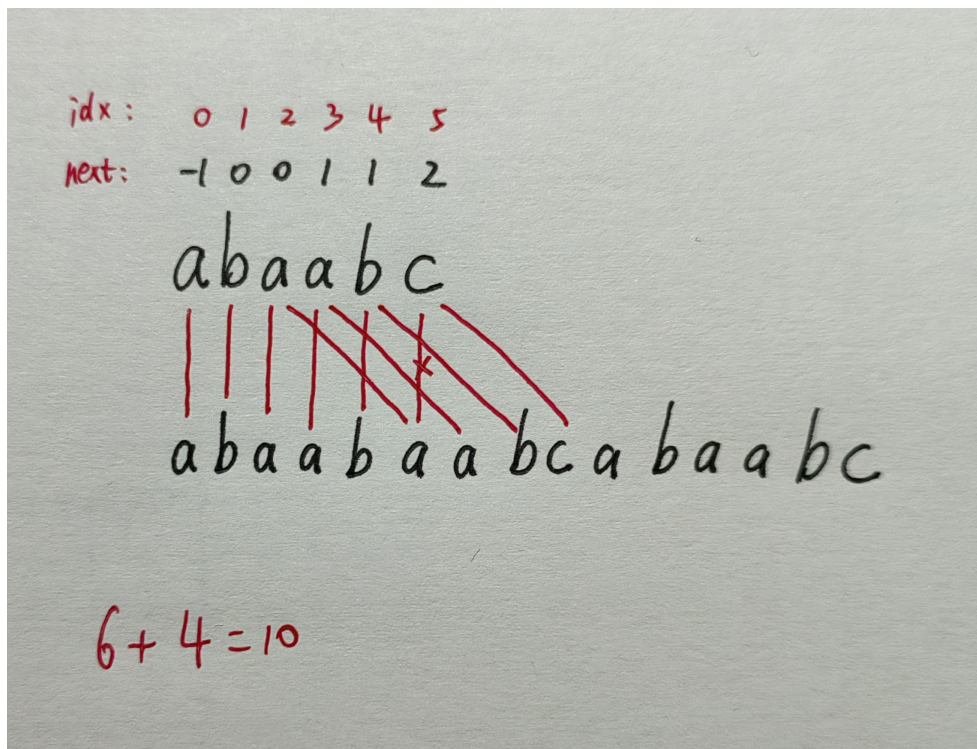
1. 首先求模式串的最长相等前后缀数组（lps 数组）

1	a	b	a	a	b	c
2	0	0	1	1	2	0

2. 整体右移一位，首位补 -1 (0-based)

1	a	b	a	a	b	c
2	-1	0	0	1	1	2

拿到 next 数组后，依次匹配文本串和模式串，失配时模式串指针回退到 next 数组指向的下标继续匹配（注意：所有都是 0-based）



10 A

第一趟快速排序后，数组中有一个元素，左边都比它小，右边都比它大

然后注意分治，第一趟主元左边子数组为一个子问题，右边子数组为一个子问题

第二趟快速排序后，左右两个子问题中也必须分别有一个元素，左边都比它小，右边都比它大

所以这种题不是单纯的找两个元素看左边小右边大这么简单

11 A

最佳归并树计算虚段个数

首先 k 路归并的最佳归并树是正则 k 叉树（哈夫曼树）

于是满足以下式子

$$\begin{aligned} n &= n_0 + n_k \\ n &= \text{度数} + 1 = kn_k + 1 \end{aligned}$$

于是有

$$n_k = \frac{n_0 - 1}{k - 1}$$

要使归并正常进行，这个式子必须整除，于是令虚段个数为 x

只要满足 $(n_0 + x - 1) \% (k - 1) = 0$ 即可

易解出 x

15 A

这个题的意思是：使用基址寻址，在基址 $F0000000H$ 上偏移 $FFFFFF12H$ ，即 $FFFFFF12H$ ，得到机器数 $1234FF00H$ 的首地址，又因为机器是大端方式，机器数高位存在低地址处，于是最低有效字节 $00H$ 地址为 $FFFFFF12H + 3 = FFFFFFF15H$

18 A

1	I1: IF ID EX M WB	
2	I2: IF ID EX M WB	# 无数据冒险
3	I3: IF	ID EX M WB # I3需要用到寄存器s2和s3，写后读
4	I4: IF ID EX M WB	# I4在I3 ID时开始，此时s3已经写入，故I4没有数据冒险

- 注意：下一条指令一般是在上一条 ID 时开始的，存在例外，例如：转移指令的下一条需要阻塞到转移指令执行 M 之后（因为转移指令在 M 阶段才写回 PC）
- 同类型题：[2023 年第 19 题](#)

19 W

bro 看着 B 写成 D 了

$$1333 \times 10^6 \times \frac{64}{8} \times 3 = 31.992 \times 10^9 B/s \approx 32GB/s$$

21 A

1s 内 IO 次数为 $\frac{50kB}{4B} = 12.5k$ 次

$$\begin{aligned} CPU \text{用于该设备输入/输出的时间占整个CPU时间的百分比} &= \frac{1sIO \text{次数} \times \text{每次IO用多少时钟周期}}{CPU \text{主频}} \\ &= \frac{12.5 \times 10^3 \times 1000}{1 \times 10^9} = 1.25\% \end{aligned}$$

同类型题：[IO 占 CPU 总时间的百分比](#)

22 W

DMA 的数据传送过程

- 预处理：CPU 通过执行设备驱动程序，设置 DMA 控制器的传送参数（如内存地址、数据长度、传送方向等）
- 数据传送：数据传送前由 DMA 控制器请求总线使用权，数据传送又 DMA 控制器直接控制总线完成
- 后处理：DMA 控制器向 CPU 发送中断请求，CPU 执行中断服务程序做 DMA 结束处理

23 A

- 多线程模型中，用户级线程和内核级线程的连接方式有一对一，多对一，多对多
只有一对一时，OS 为每个用户级线程建立一个 PCB
- 用户级线程的管理工作可以只在用户空间中进行，故可以在不支持内核级线程的操作系统上实现

25 A

操作系统不同，底层逻辑、实现方式均不相同，为应用程序提供的系统调用接口也不同

26 W

FAT 表（文件分配表）可以类比为一种“用数组实现的链表”

文件分配表（FAT）的表项与物理磁盘块一一对应，并且可以用一个特殊的数字 -1 表示文件的最后一块，用 -2 表示这个磁盘块是空闲的（当然，规定用 -3、-4 来表示也是可行的）。因此 FAT 不仅记录了文件中各个块的先后链接关系，**同时还标记了空闲的磁盘块**，操作系统可以通过 FAT 对文件存储空间进行管理

27 W

注意题目中说的：Q1 为空时，系统才会调度 Q2 中的进程

28 A

段 S 对于进程 P、P 来说，使用位置可能不同，所以在不同进程中的逻辑段号可能不同

段表项存放的是段的物理地址（包括段始址和段长度），对于共享段 S 来说物理地址唯一

33 W A

记忆题

OSI 七层模型	TCP/IP 概念层模型	功能	TCP/IP 协议族
应用层	应用层	文件传输，电子邮件，文件服务，虚拟终端	TFTP, HTTP, SNMP, FTP, SMTP, DNS, Telnet
表示层		数据格式化，代码转换，数据加密	没有协议
会话层		解除或建立与别的接点的联系	没有协议
传输层	传输层	提供端对端的接口	TCP, UDP
网络层	网络层	为数据包选择路由	IP, ICMP, RIP, OSPF, BGP, IGMP
数据链路层	链路层	传输有地址的帧以及错误检测功能	SLIP, CSLIP, PPP, ARP, RARP, MTU
物理层		以二进制数据形式在物理媒体上传输数据	ISO2110, IEEE802, IEEE802.2

34 W

- T = Twisted pair（双绞线）
- F = Fiber（光纤）

35 A

$$W_T + W_R \leq 2^n$$

这里 $n = 3$, $W_T = 5$, 解得 $W_R \leq 3$

GBN 协议 $W_R = 1$, SR 协议 $W_R > 1$

36 A

CSMA/CD 协议最小帧长计算， τ 为单向传播时延

$$\text{最小帧长} = \text{链路带宽} \times RTT = \text{链路带宽} \times 2\tau$$

37 W

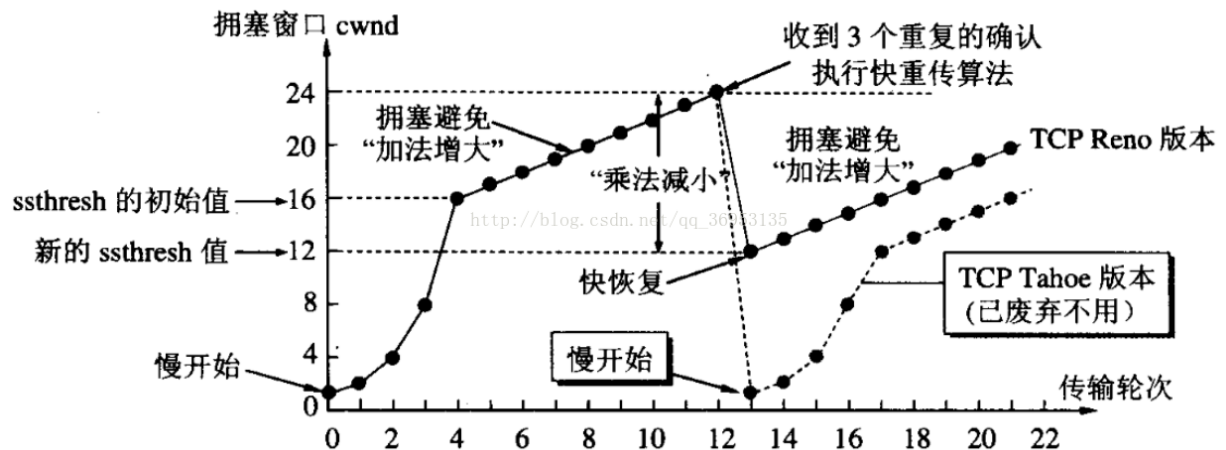
bro 这也能算错 $2^8 \neq 128$, $2^8 = 256$

38 A

TCP 规定当发送方收到对同一个报文段的 3 个重复的确认时, 就可以认为跟在这个被确认报文段之后的报文已经丢失, 立即执行快速重传算法 (注意发送该报文之前的 *ack* 不算在内)

扩展

[TCP 拥塞控制——快重传和快恢复-CSDN 博客](http://blog.csdn.net/qz_3673135)



大题

41 A

```
1 #include <bits/stdc++.h>
2
3 struct Node {
4     int val;
5     Node* next;
6 };
7
8 Node* init(const std::vector<int>& arr) {
9     if (arr.empty()) return nullptr;
10    Node* h = new Node{arr[0], nullptr};
11    Node* p = h;
12    for (int i = 1; i < arr.size(); ++i) {
13        Node* node = new Node{arr[i], nullptr};
14        p->next = node;
15        p = node;
16    }
17    return h;
18 }
19
20 void destroy(Node*& h) {
21     while (h) {
22         Node* p = h;
23         h = p->next;
24         delete p;
25     }
26     delete h;
27 }
28
29 void print(Node* h) {
30     for (Node* p = h; p; p = p->next) {
31         std::cout << p->val << ' ';
32     }
33 }
```

```

33     std::cout << std::endl;
34 }
35
36 // 链表快慢指针取中-向下取整
37 int get_mid_first(Node* h) {
38     if (!h) return -1;
39
40     Node* fast = h;
41     Node* slow = h;
42     while (fast->next && fast->next->next) {
43         fast = fast->next->next;
44         slow = slow->next;
45     }
46     return slow->val;
47 }
48
49 // 链表快慢指针取中-向上取整
50 int get_mid_second(Node* h) {
51     if (!h) return -1;
52
53     Node* fast = h;
54     Node* slow = h;
55     while (fast && fast->next) {
56         fast = fast->next->next;
57         slow = slow->next;
58     }
59     return slow->val;
60 }
61
62 // 带头结点的链表反转-迭代法
63 void rev(Node*& h) {
64     if (!h) return;
65
66     Node* prev = nullptr;
67     Node* curr = h;
68     Node* next = nullptr;
69
70     while (curr) {
71         next = curr->next; // 保存下一个节点
72         curr->next = prev; // 反转指针
73         prev = curr;      // 前移prev
74         curr = next;      // 前移curr
75     }
76
77     h = prev; // 更新头节点指向新的首节点
78 }
79
80 // 带头结点的链表反转-头插法
81 void rev2(Node*& h) {
82     if (!h) return;
83
84     Node* p = h; // 首元结点
85     h = nullptr; // 暂时清空头结点
86     Node* q = nullptr;
87
88     while (p) {
89         q = p->next; // 保存下一个节点
90         p->next = h; // 将当前节点插入到头节点后
91         h = p;
92         p = q; // 前移p
93     }
94 }
95
96 // 2019-41
97 void cross(Node* h) {
98     if (!h) return;

```

```

99
100 // 取中
101 Node* slow = h;
102 Node* fast = h;
103 while (fast->next && fast->next->next) {
104     fast = fast->next->next;
105     slow = slow->next;
106 }
107
108 // 逆置slow之后的链表
109 Node* prev = nullptr;
110 Node* curr = slow->next;
111 Node* next = nullptr;
112 while (curr) {
113     next = curr->next;
114     curr->next = prev;
115     prev = curr;
116     curr = next;
117 }
118 slow->next = prev;
119
120 // 构造新链表
121 Node* a = h;
122 Node* b = slow->next;
123 slow->next = nullptr; // 断开前后两半
124 while (a && b) {
125     Node* an = a->next;
126     Node* bn = b->next;
127     a->next = b;
128     if (an) {
129         b->next = an;
130     }
131     a = an;
132     b = bn;
133 }
134 }
135
136 int main() {
137     Node* h = init({1, 2, 3, 4, 5});
138     Node* h2 = init({1, 2, 3, 4, 5, 6});
139
140     rev(h);
141     print(h); // 5 4 3 2 1
142
143     rev2(h);
144     print(h); // 1 2 3 4 5
145
146     std::cout << get_mid_first(h) << std::endl; // 3
147     std::cout << get_mid_first(h2) << std::endl; // 3
148     std::cout << get_mid_second(h2) << std::endl; // 4
149
150     cross(h);
151     print(h); // 1 5 2 4 3
152     cross(h2);
153     print(h2); // 1 6 2 5 3 4
154
155     destroy(h2);
156     destroy(h);
157 }

```

这个题其实出的并不好，自由度太高，选取其他写法（如带尾指针的循环单链表）就不好判满了

使用带首尾指针的循环单链表

push 时只有队满才开新节点，否则赋值进 rear 所指节点，rear 后移

pop 时若队空直接返回，否则取 front 所指节点元素，front 后移

因为是循环链表，front 和 rear 可以做到类似循环数组的效果

44 W

这道题不同于盘片-磁道-扇区的查询方式，采用柱面-磁道-扇区的方式。前两个意义相反

磁盘物理地址 = 柱面号 + 磁道号 + 扇区号

一个柱面 10 个磁道，一个磁道 200 个扇区，一个簇两个扇区，易得一个柱面有 1000 个簇，于是将簇号除以 1000 即可得到柱面号（题目不严谨没给簇号从 0 开始还是 1 开始，这里默认从 0 开始）

簇号 / 每个柱面的簇数 = 柱面号 当前柱面的簇偏移量

当前柱面的簇偏移量 / 每个磁道的簇数 = 磁道号

当前磁道的起始扇区号 = 当前柱面的簇偏移量 * 2

当前磁道的起始扇区号 % 每个磁道的扇区号 = 扇区号

47 A

默认网关是本网络的出口，需要配给路由器对应本网络的接口。默认网关不是网络号，而是一台合法的主机

设备名称	能否隔离冲突域	能否隔离广播域
集线器	不能	不能
中继器	不能	不能
交换机	能	不能
网桥	能	不能
路由器	能	能

2021 年

选择题

1 W

01. 【解析】

如图 1 所示，要删除带头结点的非空单循环链表中的第一个元素，就要先用临时指针 q 指向待删结点， $q = h \rightarrow next$ ；然后将 q 从链表中断开， $h \rightarrow next = q \rightarrow next$ （这一步也可写成 $h \rightarrow next = h \rightarrow next \rightarrow next$ ）；此时要考虑一种特殊情况，若待删结点是链表的尾结点，即循环单链表中只有一个元素（p 和 q 指向同一个结点），如图 2 所示，则在删除后要将尾指针指向头结点，即 $if(p == q) p = h$ ；最后释放 q 结点即可，答案选 D。

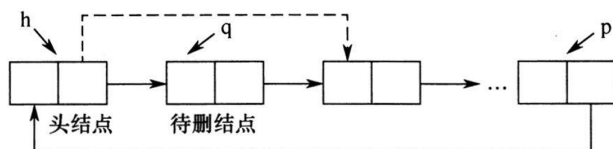


图 1

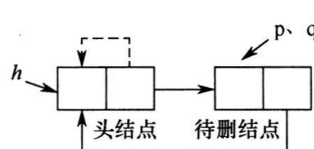


图 2

4 A

- 已知先序中序：先序为纵轴从上至下，中序为横轴从左至右
- 已知后序中序：先序为纵轴从下至上，中序为横轴从左至右
- 其他情况不能唯一确定二叉树

5 A

看到二叉树的最小带权路径长度，想到哈夫曼树

结点的带权路径长度（WPL）：从树的根结点到任意结点的路径长度（经过的边数）与该结点上权值的乘积

8 W A

粗心

dijkstra：首先初始化 dis 数组全为无穷，遍历顶点 1 所有邻接顶点 i，将距离写入 dis [i]。算法每次选择则一个到顶点 1 距离最近的顶点 j 加入顶点集 S，并判断由顶点 1 绕行顶点 j 后到任一顶点 k 是否距离更短，将到顶点 k 的距离松弛为顶点 1 到顶点 j 的距离加顶点 j 到顶点 k 的距离

本题的错误点在于没看到从顶点 5 绕行顶点 2 的路径距离更近

12 A

- M FLOPS：百万 10^6
- G FLOPS：十亿 10^9
- T FLOPS：万亿 10^{12}
- P FLOPS：千万亿 10^{15}
- E FLOPS：百京 10^{18} ，1京 = 1亿亿 = 10^{16}
- Z FLOPS：十万京 10^{21}

16 W A

粗心

本题的错误点在于错误地认为 $32 = 2^6$ ，实际上 $32 = 3^5$ 。不是哥们

18 A M

- **数据通路** 是处理器中用于执行算术和逻辑操作的部分，主要包括组合逻辑元件（如 ALU）和时序逻辑元件（如寄存器）
- 状态单元：进行数据存储（时序逻辑电路 & 指令、数据锁存器）
- 组合单元：进行运算操作（组合逻辑电路 & 与或非的电路组合）
- **数据通路包含用于异常事件检测及响应的电路**

19 A M

- 同步总线是指总线通信的双方采用同一个时钟信号，但是一次总线事务不一定在一个时钟周期内完成，即 **时钟频率不一定等于工作频率**
- 异步总线采用握手的方式进行通信，每次握手的过程完成一次通信，但一次通信往往会交换 **多位** 而非一位数据

20 W M

I/O 接口即 I/O 控制器，其功能是接收主机发送的 I/O 控制信号，并实现主机和外部设备之间的信息交换。磁盘驱动器是由磁头、磁盘和读写电路等组成的，也就是我们平常所说的磁盘本身

I/O 接口：打印机适配器、网络控制器、可编程中断控制器

23 A M

常见的特权指令：只能在内核态执行

- 有关对 IO 设备操作的指令
- 有关访问程序状态的指令
- 存取寄存器指令

- 其他指令

会使 CPU 从用户态进入内核态的指令属于非特权指令

25 A

调度程序从 **就绪队列** 中选择一个进程为其分配时间片

阻塞队列中的进程只有被唤醒进入就绪队列后，才能参与调度，所以该调度过程不使用阻塞队列

26 A 重要-差一问题

- 磁头从磁道 a 移动到磁道 b ，移动距离为 $|a - b|$ (关注的是间隔)
- 闭区间中有多少个整数 (包含端点) : $b - a + 1$ (关注的是点)
- 从 a 开始往后数 b 个数 (位移/第 b 个数) : $a + b - 1$

解决典型 off-by-one 问题 (差一问题) : 点的个数 = 步数 + 1
距离/步数关注的是“间隔” (edges), 计数关注的是“点” (vertices)

28 W A

存在位 = 0 的页根本不在内存，页框号字段是无效的，所以也无法被替换。换句话说，只有存在位 = 1 的页，才“真的”在内存中，才会参与替换算法

存在位 = 0 是什么原因导致的？

- 该页从未被访问过
- **该页之前在内存中，但被换出**，其内容已保存到外存
- 页表刚建立时

32 W M

系统调用是由用户进程发起的，请求操作系统的服务

- 页置换由操作系统自动完成，不涉及系统调用
- 进程调度完全有操作系统完成，无法通过系统调用完成
- 创建新进程可以由系统调用完成

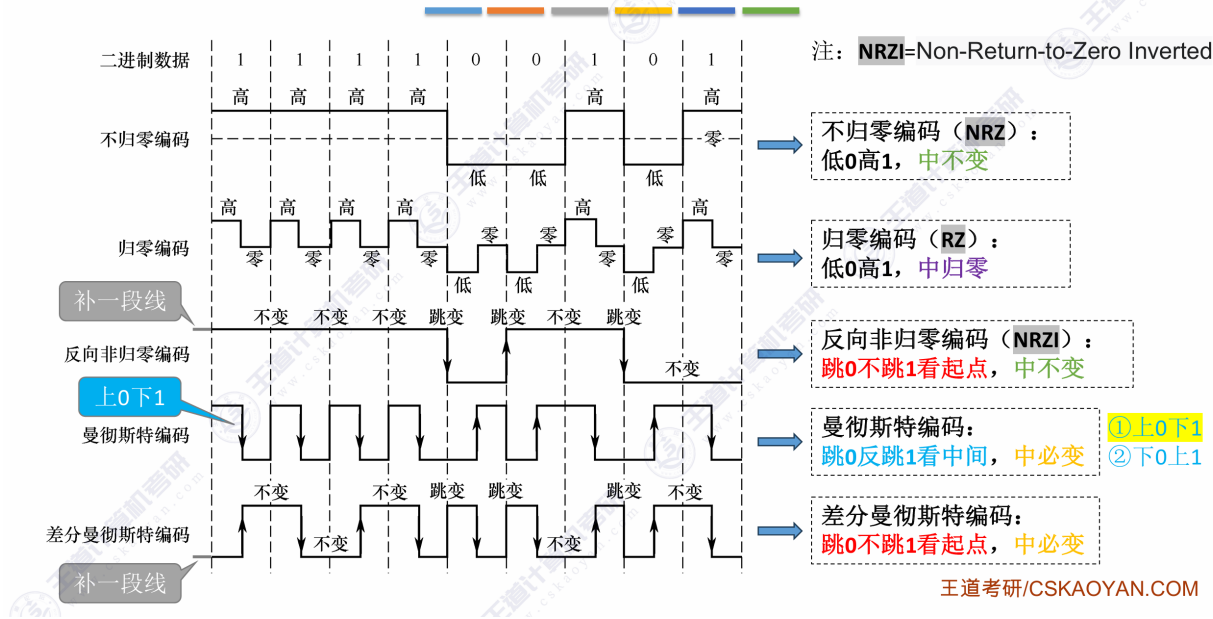
33 W M

注意题目是 TCP/IP 参考模型，即 4 层模型

端到端报文段传输是传输层的问题

传输层下一层为网络层，主要功能为路由选择

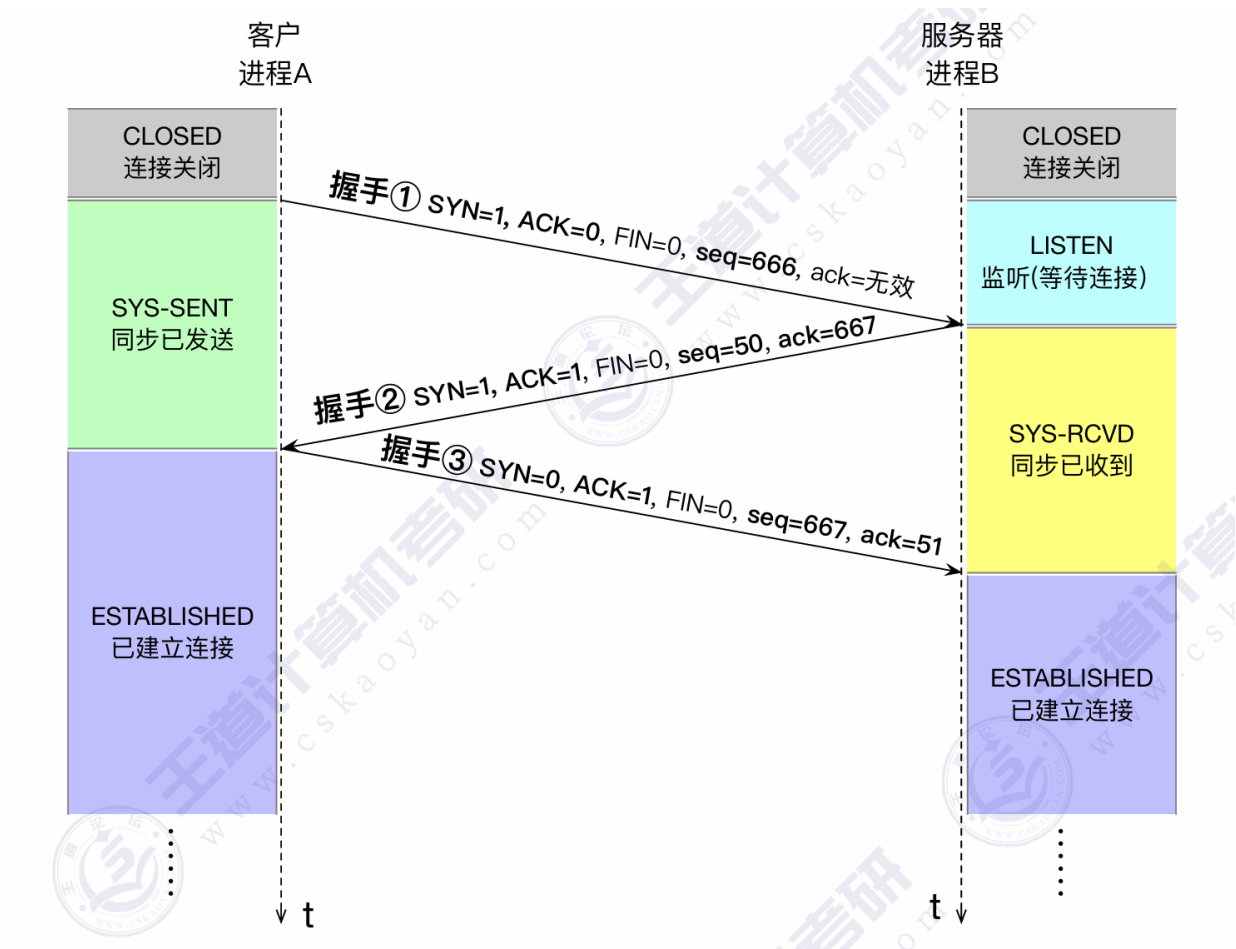
常用的编码方法



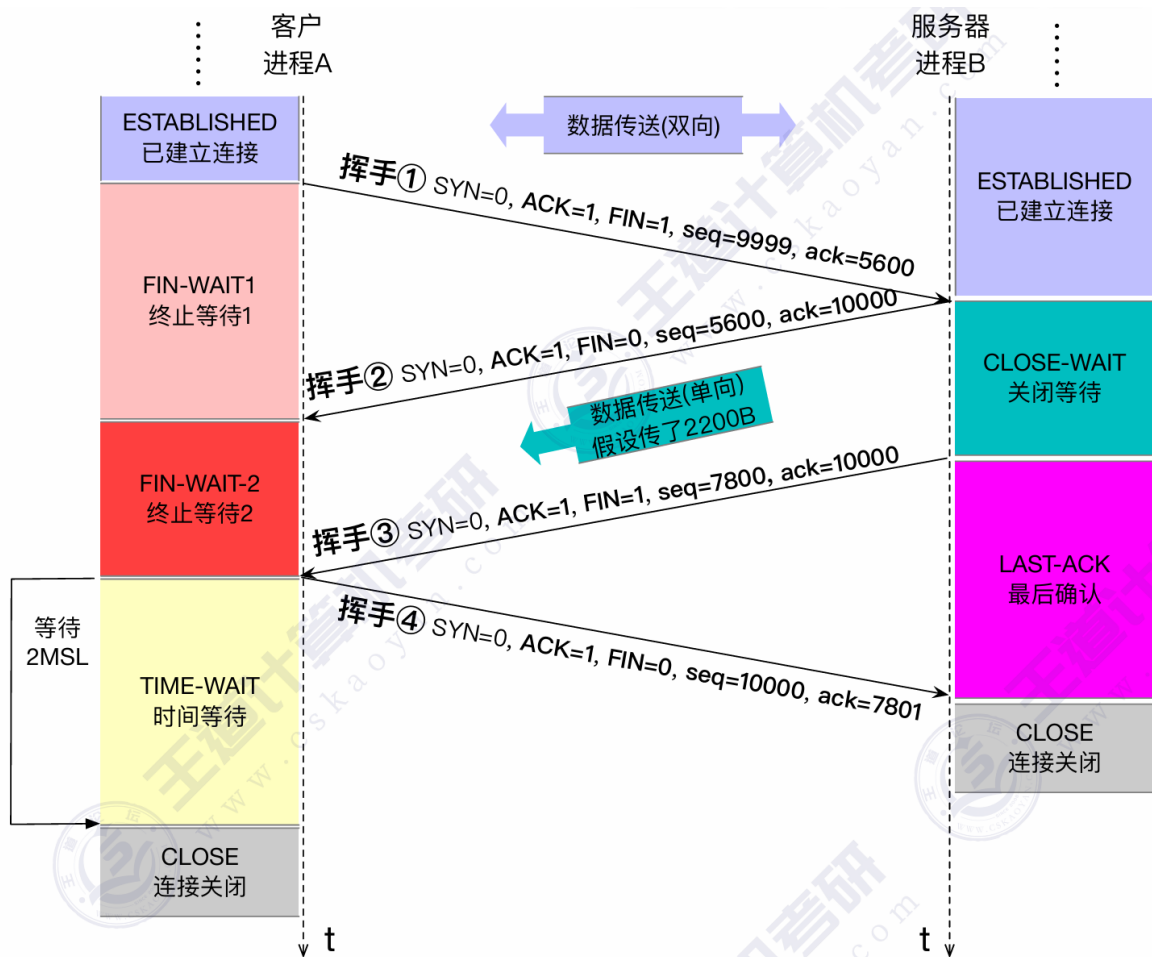
38 A M

记忆题, 注意客户端和服务端状态的变化

三次握手



四次挥手



39 A M

记忆题，关键在于记住常用协议的首部长度，题目中一般不会给

协议/层次	默认首部长度
应用层-HTTP	无固定首部
应用层-DNS	12B
传输层-TCP	20B
传输层-UDP	8B
网络层-IPv4	20B
网络层-IPv6	40B
网络层-ICMP	8B
数据链路层-ARP	28B
数据链路层-以太网帧	18B (14B 首部 + 4B 尾部)

大题

41

粗心，注意题目说的是无向图！

无向图中，节点的度等于节点邻接节点的个数

若采用有向图的算法来计算入度和出度（虽然无向图中没有“入度”和“出度”的概念），计算出的结果是正确值的 2 倍

于是这个算法将给出错误的结果，因为按照算法所有节点的度都将为偶数

纠正错误后，我们可以在第一层循环里设置 deg 变量，只要 deg 为奇数就计数，这样就可以将空间复杂度优化为 $O(1)$

```

1  int IsExistEL(MGraph G) {
2      int cnt = 0;
3      for(int i = 0; i < G.numVertices; i++) {
4          int deg = 0;
5          for(int j = 0; j < G.numVertices; j++) {
6              deg += G.Edge[i][j];
7          }
8          if(deg & 1) {
9              cnt++;
10         }
11     }
12     return cnt == 0 || cnt == 2;
13 }

```

42

无错误

43

1. ALU 的宽度即 ALU 运算对象的宽度，通常与字长相同

MAR 的宽度取决于定长指令字的宽度

MDR 的宽度取决于数据线宽度

2. 一般来说，指令格式能定义 $2^{\text{操作码位数}}$ 种操作

本题 R 型格式操作码为 4 位，于是有 $2^4 = 16$ 种操作

I 型和 J 型均有 6 位操作码，共享这 6 位操作码空间，特别的，000000 被 R 型指令占用，于是共有 $2^6 - 1 = 63$ 种操作

44

三种映射方式中 Tag 的用途

- 直接映射：用于唯一标识主存块，直接映射只需比较一个 Tag 即可判断命中与否
- 全相联映射：用于唯一标识主存块，全相联映射必须比较所有 Tag 才能知道命中与否
- 组相联映射：用于区分同一组中不同主存块

当访问一个主存块时：

- 根据 **组号** 找到对应的一组
- 比较该组中每个 Cache 行存储的 **Tag**
 - 若有一个 Tag 相同且有效位为 1 → 命中
 - 若两个 Tag 都不匹配 → 未命中（需要替换，此时需要替换算法出场）

本题采用 LRU 替换策略，根据最近最少使用原则，12 刚被访问过，应该淘汰 4

45

2) 方法 1 错误。在 wait() 中，当 $S \leq 0$ 时，关中断后，其他进程无法修改 S 的值，while 语句陷入死循环。方法 2 正确。方法 2 在循环体中有一个开中断操作，这样就可以使其他进程修改 S 的值，从而避免 while 语句陷入死循环。

46

无错误

47

1. 对于有线网络：现代交换式以太网的数据链路层已不再使用 CSMA/CD。CSMA/CD 只与早期的共享式总线以太网相关

对于无线网络：其数据链路层遵循 IEEE 802.11 标准，核心介质访问控制协议就是 CSMA/CA

本题的网络拓扑结构正好属于早期的共享式总线以太网，于是数据链路层使用的 CSMA/CD 协议

在 408 考试的语境下，当不涉及更基础的随机访问协议时，**通常默认有线局域网的数据链路层介质访问控制协议就是 CSMA/CD**

所谓更基础，指的是信道划分介质访问控制、ALOHA 协议和 CSMA 协议

- 2. 在整个过程中，并没有涉及 H2，H2 没有主动交换数据
- 3. H2 会收到两个 ARP 帧，第一次是 H1 希望查询本地域名服务器的 MAC 地址，第二次是 H1 访问 Web 服务器前希望查询路由器 R 的 MAC 地址

2022 年

选择题

3 W

bro 排除了 1 4 就认定 2 3 一定全是对的？

5 W A

对于有限字符集 S ，用二叉树表示 S 的哈夫曼编码集和定长编码集，分别得到二叉树 T1 和 T2，我们有以下结论

- T1 与 T2 的节点数不一定相同
- 出现频次不同的字符在 T1 中可能处于相同的层
- 所有字符一定都在 T2 中处于相同的层，而且都是叶子结点

6 A

对于无向图，至少要有 $|V| - 1$ 个顶点才可能连通，顶点数小于 $|V| - 1$ 一定不可能连通

8 A

老生常谈的共识

对于 m 阶 B 树，节点最小度 $t = \lceil m/2 \rceil$

- 对于根节点（不是叶子），关键字数取值范围为 $[1, m - 1]$
- 对于非根节点，关键字数取值范围为 $[t - 1, m - 1]$

关键字数 = 孩子数 - 1

扩展：B 树与 B+树的异同

1. 相同点

特性	B 树	B+ 树	说明
阶数	m	m	节点最多 m 个孩子
最小度	$t = \lceil m/2 \rceil$	$t = \lceil m/2 \rceil$	最小度 t 约束节点最少关键字/孩子数
非根节点关键字数	$[t - 1, m - 1]$	$[t - 1, m - 1]$	内部节点遵循相同规则
根节点关键字数	$[1, m - 1]$ (非叶根)	$[1, m - 1]$ (非叶根)	根节点特例相同

2. 不同点

特性	B 树	B+ 树	说明
数据存储	内部节点 + 叶子节点	只在叶子节点	内部节点只存索引
叶子节点	不一定连成链表	叶子节点形成链表	B+ 树支持范围查询
查询效率	查找路径即数据	查找路径到叶子才取数据	B+ 树更适合范围查询和磁盘存储

10 W

送分概念题

归并的含义：将两个或两个以上的有序表合并成一个新的有序表
二路归并是将两个有序表合并为一个新的有序表

15 W

题目中的虚页号没加后缀 H ，是十进制！老头玩阴的

16 A

Cache 比较器的个数就是分组中的 Cache 行数

Cache 比较器的位数就是 Tag 标记位数（Tag 的作用就是组内区分 Cache 行）

17 A

内存条的容量为 512MB，于是地址需要 29 位

芯片内行数是 8192，一行的大小是 8192x8bit，行缓冲长度就是一行的大小

18 W A

指令集处于软硬件的交界面上

指令字和指令格式、通用寄存器个数和位数都与机器指令有关，由 ISA 规定

两个 CPU 可以有不同的时钟周期，但指令集可以相同

CPU 的时钟周期不由 ISA 规定。加法器的进位方式涉及电路设计，也不由指令集规定

19 W A

地址码为 6 位，一条二地址指令会占用 2^6 条一地址指令的空间，一条一地址指令会占用 2^6 条零地址指令的空间

如果全部是零地址指令，最多有 2^{16} 条

于是减去一地址指令和二地址指令所占用的零地址指令空间，即 $2^{16} - 254 \times 2^6 - 12 \times 2^6 \times 2^6 = 128$ 条

20 W A

将源程序转换为可执行目标文件的过程分为预处理、编译、汇编、链接四个阶段

21 A

缓冲区每写满一次，就会向 CPU 发出一次中断请求，CPU 响应并处理中断的过程，就是将缓冲区中的数据“取走”的过程

22 W A

MIMD 结构分为多计算机系统和多处理器系统

- 向量处理器是 SIMD 的变体，属于 SIMD 结构
- 硬件多线程技术是在一个核中处理多个线程，可用于单核处理器
- 共享内存多处理器(SMP)具有共享的单一物理地址空间，所有核都可通过存取指令来访问同一片主存地址空间

24 A

在操作系统初始化的过程中需要创建中断向量表，用于实现“中断处理”，CPU 检测到中断信号后，根据中断号查询中断向量表，跳转到对应的中断处理程序

硬盘分区表是在硬盘逻辑格式化时创建的

文件系统的根目录、文件系统的索引结点表都是在文件系统初始化时创建的

25 W

0 时刻调度进程 P0 获得 CPU 也要算在内，一共 6 次进程调度

31 W A

发生系统调用时，CPU 执行陷入 (Trap) 指令

检测到“内中断”后，由 CPU 负责保存断点 (PC) 和程序状态字，并将模式改为内核态

然后执行操作系统内核的系统调用入口程序，该内核程序负责保存通用寄存器的内容，再调用某个特定的系统调用服务例程

特点	内中断 (Trap)	外中断 (Interrupt)
触发方式	CPU 执行特定指令 (如 syscall)	外部设备或硬件信号
可预测性	可预测	不可预测
CPU 保存现场	自动保存 PC/PSW	自动保存 PC/PSW
内核保存寄存器	系统调用入口程序保存	中断入口程序保存
中断向量	固定为系统调用入口	由中断向量表指定

扩展

关于中断隐指令

- **内中断 (Trap) :**
 - 指令本身就是显式触发内中断的指令，如 `INT n`、`SYSCALL`
 - 没有所谓的“隐指令”，CPU 直接识别指令并触发内核入口
- **外中断 (External IRQ) :**
 - 设备中断是硬件发出的信号，CPU 并没有执行特定指令
 - 因此 CPU 需要 **中断硬件逻辑/控制逻辑** 来响应这个外部请求，有时候称为 **隐式中断指令 (implicit trap logic)**，但严格来说，它不是用户程序执行的指令，而是 **CPU 内部硬件逻辑自动触发的异常流程**
 - CPU 会在硬件检测到中断信号后，自动完成 PC/PSW 保存和跳转

简单理解：

- 内中断：程序执行的 **显式指令**
- 外中断：CPU 硬件 **隐式触发**，无需程序指令

32 W A

当一个设备被连接到主机时，驱动程序负责初始化设备（如将设备控制器中的寄存器初始化）

当进程在执行驱动程序时，可能会因为设备忙碌而进入阻塞态

设备的 I/O 操作本质就是在设备控制器和主机之间传送数据，而只有厂家知道设备控制器的内部实现，因此也只有厂家提供的驱动程序能控制设备的 I/O 操作

厂家会根据设备特性，在驱动程序中实现一种合适的 I/O 控制方式，故驱动程序与 I/O 控制方式有关

34 W A

使用奈奎斯特定理

$$\text{最大数据传输率} = 2B \log_2 M$$

4 个幅值的 ASK 调制说明有 4 个相位

$M = 4$ ，于是得到 $800kbps$

定理/公式	最大速率公式	参数含义	单位
奈奎斯特定理	$C = 2B \log_2 M$	B : 信道带宽 (Hz) ; M : 每符号电平数	bps

定理/公式	最大速率公式	参数含义	单位
二进制特例	$C = 2B$	B : 信道带宽 (Hz) ; $M = 2$	bps
香农公式	$C = B \log_2(1 + S/N)$	B : 信道带宽 (Hz) ; S/N : 信噪比 (线性)	bps

36 A

默认网关可以理解为离当前主机最近的路由器的端口地址

主机的子网掩码与它的网关的子网掩码相同

37 W A

SDN 对上层开发者提供的编程接口称为 **北向接口**，而 **南向接口** 则负责控制平面和数据平面间的通信，所以 SDN 控制器向数据平面的 SDN 交换机下发流表时使用南向接口

38 W

时刻 0 发生了超时，门限值 ssthresh 变为拥塞窗口的一半即 8，同时 cwnd 置为 1，执行慢开始算法，cwnd 指数增长，达到 ssthresh 后面，执行拥塞避免算法，cwnd 线性增长

时刻	0	1	2	3	4	5	6	7	8	9	10	11
拥塞窗口	1	2	4	8	9	10	11	12	13	14	15	16

大题

42

100000 个数中找出最小的 10 个数：使用长度为 10 的大根堆，遍历数组，若当前值比堆顶小则弹出堆顶并将当前值插入堆中，直到遍历完成

时间复杂度：O(n)

空间复杂度：O(1)

43

第一问思路是对的，但是要注意逻辑表达式的格式

1) 符号标志 SF 表示运算结果的正负性，因此 $SF = F_{15}$ 。

对于加法运算 $A + B \rightarrow F$ ，若 A、B 为负，且 F 为正，则说明发生溢出；或者，若 A、B 为正，且 F 为负，也说明发生溢出。因此，加运算时，溢出标志 $OF = \overline{A_{15}} \cdot \overline{B_{15}} \cdot F_{15} + A_{15} \cdot B_{15} \cdot \overline{F_{15}}$ 。

对于减法运算 $A - B \rightarrow F$ ，若 A 为负、B 为正，且 F 为正，则说明发生溢出；或者，若 A 为正、B 为负，且 F 为负，也说明发生溢出。因此，减运算时，溢出标志 $OF = \overline{A_{15}} \cdot B_{15} \cdot F_{15} + A_{15} \cdot \overline{B_{15}} \cdot \overline{F_{15}}$ 。

第三问，rd 表示寄存器编号，应连接地址译码器

扩展

地址译码器与多路选择器

- **译码器**：快速“点亮”目标寄存器，就像 linux 系统编程中的位掩码 flags 导向某个模式
- **MUX**：每条输入都存在，通过选择逻辑输出，就像 switch-case 逐个判断分支

44

磁盘属于冷门知识，牢记概念很重要

磁盘物理地址：柱面号、磁头号、扇区号

- 柱面号：每个盘面的磁道数取 log
- 磁头号：盘面数取 log

- 扇区号：每个磁道的扇区数取 $\log$

扇区访问时间：寻道时间 + 平均延迟时间 + 平均传输时间

- 寻道时间：题目给了
- 平均延迟时间：磁盘转半圈所用的时间
- 平均传输时间：一个扇区划过磁头下方所需要的时间 = 磁盘转一圈的时间 / 磁道的扇区数

45

由图可知，文件 doc 和文件 course1 对应的索引结点号都是 10。说明 doc 和 course1 两个目录项共享同一个索引结点，本质上对应同一个文件。而文件 course1 存储在 30 号磁盘块，因此文件 doc 占用的磁盘块的块号 x 为 30

47

假设 H2 与 H3 之间的最远距离是 D ，根据 CSMA/CD 协议的工作原理有

$$\text{最短帧长} = \text{总线传播时延} \times \text{数据传输速率} \times 2$$

使用 100BaseT 局域网标准，所以数据传输速率为 100Mbps

总线传播时延由两部分组成，一部分是信号传播时延，另一部分是信号通过设备 2 时产生的额外 $1.51\mu\text{s}$ 时间延迟

$$\text{信号传播时延} = \frac{D}{\text{信号传播速度}} = \frac{D}{2 \times 10^8 \text{m/s}}, \text{ 带入方程求解即可}$$

// TODO

2023 年

选择题

9 A

0	1	2	3	4
	2025	12		25(删除标记)
1	3	2	1	2

$$ASL_{fail} = \frac{1 + 3 + 2 + 1 + 2}{5} = 1.8$$

线性探测再散列法中删除一个关键字会导致后面的关键字无法通过线性探测找到正确的位置。当删除一个关键字时，为了保持散列表的连续性，通常会将后续的关键字向前移动填充空缺，这样后续的查找操作才能继续正确地找到它们。然而，如果删除的是一个位于中间位置的关键字，后面的关键字需要依次向前移动，这会导致删除操作的时间复杂度较高，因为需要移动大量的关键字。为了解决删除操作中的位置依赖性问题，可以使用 **删除标记** 来表示一个位置上的关键字已被删除。

散列表长度为 n ，散列函数 $H(k) = k \% m$ ，插入 t 个元素

- $ASL_{success}$ ：在构建散列表时表明每个元素的查找长度 a_i ， $ASL_{success} = \frac{1}{t} \sum_{i=1}^t a_i$
- ASL_{fail} ：需要分析散列表的每个位置，注意取模
 - 若没有元素，查找长度为 1
 - 若有元素，需要判断该元素到最近的空节点的距离（从空节点 1 开始数，包含自身和空节点）
 - 若该元素已被删除（del），也需要判断该元素到最近的空节点的距离（可能循环访问，同上）

7 W M

B 树删除非叶节点元素最终都转换成删除叶结点元素

12 A

主频 = 1.5GHz 表示 CPU 时钟周期为 $\frac{1}{1.5 \times 10^9} s$

CPI = 1.2 表示 每条指令执行平均需要 1.2 个时钟周期

指令执行速度（每秒执行多少条指令） = $1.5G/1.2 = 1.25G$ 条

用户 CPU 时间（执行程序 P 所用时间） = $\frac{1}{1.5 \times 10^9} \times 1.2 \times 5 \times 10^5 = 0.4ms$

14 W

80200000H 阶码全零，符号位为 1，位数为 0100..., 为非规格化负数

于是指数为 -126，尾数形式为 0.M 而不是 1.M

故结果为 $-0.01 \times 2^{-126} = 2^{-128}$

值的类型	单精度 float (32位)				双精度 double (64位)			
	符号	阶码	尾数	值	符号	阶码	尾数	值
正零	0	全0	全0	+0	0	全0	全0	+0
负零	1	全0	全0	-0	1	全0	全0	-0
非规格化正数	0	全0	$f \neq 0$	$2^{-126}(0.f)$	0	全0	$f \neq 0$	$2^{-1022}(0.f)$
非规格化负数	1	全0	$f \neq 0$	$-2^{-126}(0.f)$	1	全0	$f \neq 0$	$-2^{-1022}(0.f)$
正无穷大	0	全1	全0	$+\infty$	0	全1	全0	$+\infty$
负无穷大	1	全1	全0	$-\infty$	1	全1	全0	$-\infty$
无定义数（非数）	0或1	全1	$f \neq 0$	NaN	0或1	全1	$f \neq 0$	NaN

15 W A

法一：地址空间 2^{30} ，可以将高 2 位的 00, 01, 10 状态分给 RAM，于是 ROM 地址范围为 1100..0 ~ 1111..1，即 $30000000H \sim 3FFFFFFFH$

法二：本题说存储芯片要占满所有可能的存储地址空间，并且 ROM 在高地址区，故 ROM 最大地址为 $2^{30} - 1$ ，即 $3FFFFFFFH$ ，据此也能判断选 C

法三：地址空间 2^{30} ，RAM/ROM 占用比为 3 : 1，且 RAM 在低地址，于是有 $2^{30} \times \frac{3}{4} = \frac{3}{4} \times 4 \times 2^{28} = 3 \times 2^{28} = (3000\ 0000)_{16}$ ，于是 RAM 地址范围为 $0000\ 0000H \sim 2FFF\ FFFFH$ ，同理 $2^{30} \times \frac{1}{4} = 2^{28} = (1000\ 0000)_{16}$ ，于是 ROM 地址范围为 $3000\ 0000H \sim 3FFF\ FFFFH$

16 W

法一：计算 OF 时要当成有符号数计算，而 CF 表示无符号整数运算时的进位/借位，因此计算 CF 时要当成无符号数计算 $x = 10, y = -20$ ，当成无符号数时 x 不够减 y ，故 $CF = 1$ 。此外，OF 显然为 0

法二：管他七的八的，直接转补码算，公式 $OF = C_n \oplus C_{n-1}, CF = C_{out} \oplus Sub$

```
1
2      0000 1010
3 + 1110 1100 // Sub = 1, Cn = 0, Cn-1 = 0, Cout = Cn = 0
4 -----
5      1111 0110
6 OF = 0 ^ 0 = 0;
7 CF = 0 ^ 1 = 1;
```

18 W A M

- 1. **组合逻辑元件** 是那些仅由组合逻辑电路组成的元件，其输出仅取决于当前的输入，而不受存储器或时钟信号的影响。算术逻辑部件 ALU 和多路选择器 MUX 都属于组合逻辑元件，它们的输出仅由当前输入决定
- 2. 程序计数器 PC 和通用寄存器是 **时序逻辑元件**，也称为状态元件。它们的输出不仅取决于当前输入，还受存储器或时钟信号的影响，并且具有状态或存储功能

19 W A S

```
1 I1: IF ID EX M WB # 一般计算指令在EX拿到数据
2 I2:     IF ID EX M WB # 通过旁路, 将I1的EX产生的数据转发到I2的EX
3 I3:     IF ID EX M WB # I2是load, 此时是load-use。load指令在M阶段才拿到数据, 故I3的EX阻塞到I2的M执行完后
4 I4:     IF ID EX M WB # I3是转移指令。转移指令在M阶段才写入PC
```

取指 IF、译码/取数 ID、执行 EX、访存 MEM、写回 WB

旁路技术将结果从执行阶段的功能单元 (如 ALU) 直接转发给需要使用该结果的指令, 绕过了中间的写回阶段

1. 旁路技术允许将结果从 EX 或 MEM 阶段直接转发给需要数据的指令的 EX 或 ID 阶段
2. 但对于 **load 指令** 后立即使用数据的情况 (如 load 后接分支或算术指令), 如果后续指令在 ID 阶段就需要数据, 则必须阻塞直到 MEM 阶段结束
3. 控制冒险需要通过阻塞或预测处理, 本题采用硬件阻塞, 分支指令后的指令会被阻塞直到分支决策完成
4. 旁路技术一般针对 EX 阶段, 在 load 上用旁路针对 M 阶段

只有 load 指令在 MEM 阶段才产生数据, 所以仅考虑 load 指令只能在访存阶段旁路取数这一特例

20 W A

总线数据传输过程中, 需要同时考虑数据传输和地址传输, 后者常常隐含在题目中或题目说明忽略不计。本题需要考虑以下三个时间

- 地址传输
- 数据准备
- 数据传输

每个时钟周期 $10^{-9}s$, 准备数据 $6ns$, CPU 通过地址线传输地址 $1ns$, 主存传输数据到 CPU $1ns$, 一共 $6 + 1 + 1 = 8ns$

32B 的数据在 64b 的总线上需要传输 $\frac{32 \times 8}{64} = 4$ 次, 于是总时间 $4 \times 8 = 32ns$

二刷: 不支持突发传送, 说明每次 CPU 想读取 64b 数据都要给出地址, 于是每次访存时间为
传地址 + 准备数据 + 传数据 = $1 + 6 + 1ns = 8ns$, 重复 4 次即 $32ns$

21 W S M

1. 异常在指令执行过程中检测
2. 中断请求信号在指令执行完毕后检测
3. 外设向 CPU 发送中断请求信号而不是中断结束信号: **中断请求信号总是从外设发向 CPU, 中断结束信号总是从 CPU 发向中断控制器**

二刷: C 选项也是错的, 即使开中断, 受到中断屏蔽字影响不会立即中断
这样看的话, 题出错了, CD 均可选

26 W A S

注意审题, 题目说的是下列操作 **完成** 时, 执行系统调用完成时 CPU 从内核态转为用户态

二刷:

- A. 阻塞进程完成时, CPU 仍可能处于内核态, 因为可能要执行进程调度
- B. 执行进程调度完成时, CPU 仍可能处于内核态, 因为若下一个进程是内核进程, CPU 将保持内核态
- C. 唤醒进程完成时, CPU 通常处于内核态, 因为需要继续处理中断或进程调度
- D. 执行系统调用完成时, CPU 一定出于用户态

30 A

页表完成虚拟地址到物理地址的转换, 两进程共享数据 data 的情况, 二者虚拟地址可能不相等, 但 data 被映射到物理内存的同一页框中

这是因为共享的数据页被映射到相同的物理页框中, 不同进程的虚拟地址映射到相同的物理地址

31 W A S M

索引节点是指文件系统中的一种数据结构，每个索引节点保存了文件系统中的—个文件系统对象的元信息数据，但不包括数据内容或者文件名。内存索引节点是存放在内存中的索引节点，文件被打开时，需要将磁盘索引节点复制到内存索引节点中。因此本题进程 P 关闭 F 时，需释放 F 的索引节点所占的内存空间

32 W A S

设备分配需要考虑的是

1. **设备的类型**：不同类型的设备有不同的特性和接口，需要根据设备类型进行适当的分配和管理
2. **设备的访问权限**：不同的进程可能对设备的访问有不同的权限要求。设备分配时需要考虑进程的访问权限，确保只有具有适当权限的进程能够访问和使用设备
3. **设备的占用状态**：需要考虑设备的当前使用状态，即设备是否已经被其他进程占用或者正在执行某个操作，设备分配时需要避免资源冲突
4. **逻辑设备与物理设备的映射关系**：在系统中，逻辑设备和物理设备之间存在映射关系。逻辑设备是用户程序或操作系统中对设备的抽象表示，而物理设备是实际的硬件设备。设备分配需要确保逻辑设备与物理设备之间的正确映射

二刷：不能被设备独立性软件干扰，以为有了数据独立性软件就不必考虑逻辑设备与物理设备的映射关系
相反，设备分配需要调用设备独立性软件来管理逻辑设备与物理设备的映射关系

33 W S

$$\text{传输时延} = \frac{1000 \times 8b}{100 \times 10^6 bps} = 0.08ms$$

$$\text{传播时延} = \frac{1000b}{100 \times 10^6 bps} = 0.01ms$$

$$\text{一共} \frac{1MB}{1000B} = 10^3 \text{ 个分组}$$

$$\text{画分组时空图可知总时间为 } 0.08 \times 10^3 + 0.02 + 0.08ms = 80.10ms$$

本题错就错在没有计算路由器转发的传输时延 $0.08ms$

二刷：很好，又错在设计路由器转发的传输时延

34 W A S

某 **无噪声理想信道** 带宽为 $4MHz$ ，采用 QAM 调制，若该信道的最大数据传输速率是 $48Mb/s$ ，则该信道采用的 QAM 调制方案是 ()。

- A. QAM-16
- B. QAM-32
- C. QAM-64
- D. QAM-128

奈奎斯特定理和香农公式完全忘了

对于无噪声理想信道，信道的最大数据传输速率可以通过奈奎斯特定理计算

根据奈奎斯特定理，波特率（符号传输速率） $= 2W = 2 \times 4MHz = 8MHz$ ，即每秒传输 $8M$ 次

又最大数据传输率为 $48Mbps$ ，则每次需要传输的比特数量为 $\frac{48Mbps}{8MHz} = 6b$

采用的 QAM 调制方案为 $2^6 = 64$ ，即 QAM-64

QAM：调幅（AM）与调相（PM）结合，形成叠加信号，若有 m 种振幅， n 种相位，则 1 码元 $= \log_2(mn)bit$

QAM-64：采用 QAM 调制技术，有 64 种码元

关于奈奎斯特定理和香农公式，前者适合无噪声、带宽有限的信道，后者适合有噪声、带宽有限的信道

- 奈奎斯特定理 $2W$ 计算得出的 **极限波特率** 指每秒最大符号传输次数
- 香农公式 $W \log_2(1 + S/N)$ 计算得出的 **极限比特率** 指每秒传输的最大比特数

35 A S

停止-等待协议的信道利用率公式为 $\frac{T_D}{T_D + RTT + T_A}$

GBN 协议和 SR 协议的信道利用率公式均为 $\frac{WT_D}{T_D + RTT + T_A}$

于是 U_1 最小

GBN 协议和 SR 协议二者的比较：主要看窗口大小谁比较大，令 n 为帧序号位数

- GBN 协议的发送窗口尺寸 W 范围： $[1, 2^n - 1]$
- SR 协议的发送窗口尺寸 W 范围： $[1, 2^{n-1}]$

题目给定 $n = 3$ ，显然信道利用率 $GBN > SR$

二刷：信道利用率 = 有效发送数据所占用的时间 / 整个发送周期

36 A

CSMA/CD 协议发送方发送过程中检测到冲突后会随机等待一段时间，时间间隔使用二进制指数退避算法计算，设争用期时长为 m ，这是第 k 次冲突

$$\text{随机数 } r \in [0, 2^{\min(k, 10)} - 1]$$
$$\text{等待时间 } t = rm$$

38 W

在 NAT（网络地址转换）中，路由器选择转换后的 IP 地址时，通常从配置的公共 IP 池中动态选取或静态映射，地址池可能包含路由器接口 IP 或专用公共 IP 地址

本题中，路由器可用公共 IP 地址只有两个（一头一尾分别被网络号和广播号占用），并且 195.123.0.34 已被接口占用，因此 NAT 必须使用剩余可用地址 195.123.0.33 作为转换地址，以确保 IP 分组在 Internet 上可路由

39 A

168.16.84.24/20 展开后为 168.16.01010100.24

于是最小为 168.16.01010000.00000001 = 168.16.80.1

最大为 168.16.01011111.11111110 = 168.16.95.254

大题

42

// TODO

43

// TODO

44

// TODO

46

// TODO

47

// TODO

选择题

4 A

邻接多重表

1. 适用于无向图的存储
2. 画法:
 1. 列出所有节点，标上节点索引，每个节点对应一个表头结点，表头结点有 `data`（存点权）和 `firstedge`（类似 next 指针）
 2. 对每个节点，在表头结点右侧画出邻接边
 - 边结构包含 5 个字段：`mark`（遍历标记，一般省略）、`ivex`（边的一个顶点）、`ilink`（指向下一条依附于顶点 `ivex` 的边）、`jvex`（边的另一个顶点）、`jlink` 指向下一条依附于顶点 `jvex` 的边
 3. 构建链接体系。对每个节点，从第一条边开始，将 `ilink` 指向下一条依附于顶点 `ivex` 的边，将 `jlink` 指向下一条依附于顶点 `jvex` 的边（像穿针一样）
3. 从顶点的表头结点出来，沿 `ilink` 或 `jlink` 遍历，可以快速的遍历该顶点的所有邻接顶点

十字链表

1. 适用于有向图的存储
2. 画法:
 1. 列出所有节点，标上节点索引，每个节点对应一个表头结点，表头结点有 `data`（存点权）、`firstin`（首个弧头节点，这一行都相同）、`firstout`（首个弧尾节点）
 2. 对每个节点，在表头结点右侧画出邻接边
 1. 边结构包含 4 个字段：`tailvex`（弧尾节点索引）、`headvex`（弧头节点索引）、`hlink`（指向下一个具有相同弧头的边），`tlink`（指向下一个具有相同弧尾的边）
 3. 构建链接体系。对每个节点，本行的弧尾都相同，直接从 `firstout` 出发链接起来。从 `firstin` 出发，链接所有弧头相同的边

12 A

$$\begin{aligned} i &= (32777)_{10} = (1000\ 0000\ 0000\ 1001)_2 = (0000 \cdots 1000\ 0000\ 0000\ 1001)_{\text{补}} \\ si &= (1000\ 0000\ 0000\ 1001)_{\text{补}} && \text{截断，变为负数} \\ j &= (1111 \cdots 1000\ 0000\ 0000\ 1001)_{\text{补}} = (-0111\ 1111\ 1111\ 0111)_2 = (-(32767 - 8))_{10} = (-32759)_{10} && \text{扩展符号位} \end{aligned}$$

13 A

- 机器语言：计算机能识别的二进制代码，可以 **直接执行**，一般不用机器语言直接写程序
- 汇编语言：用助记符表示的机器语言，需要翻译成机器语言才能执行
- 机器指令：CPU 能 **直接识别并执行** 的指令，它的表现形式是二进制编码
- 汇编指令：是汇编语言中使用的一些操作符和助记符，还包括一些伪指令（如 `assume`，`end`），汇编指令同机器指令——对应
- 微指令：是指在微程序控制的计算机中，同时发出的控制信号所执行的一组微操作。微指令是由同时发出的控制信号的有关信息汇集起来形成的。将一条指令分成若干条微指令，按次序 **执行** 就可以实现指令的功能。若干条微指令可以构成一个微程序，而一个微程序就对应了一条机器指令

16 A M

- Cache-主存层次的交换单位为 **主存块**，主存-外存层次交换单位为 **页**
- 硬件实现的运行速度比软件实现的运行速度要快的多，对于 Cache-主存层次交换需要快速运行的应用场景，其替换算法必须由 **硬件** 实现。主存-外存层次替换相比前者要慢得多，所以可以由 **软件** 实现，软件实现虽然比硬件实现要慢得多，但也更加灵活
- Cache-主存层次的交换运行速度很快（为纳秒级别）且交换数据量很小（交换单位为主存块，大小 64 字节或 128 字节等），其写策略 **既可以用全写法也可以用回写法**，主存-外存层次交换运行速度相比前者要慢很多（为毫秒级别）且交换数据量相比前者要大很多（交换单位为页，典型的页大小为 4KB），因此通常采用 **回写法** 写策略

- Cache-主存层次的交换运行速度很快（为纳秒级别）且交换数据量很小（交换单位为主存块，大小为 64 字节、128 字节等），如果采用直接映射方式，考虑最坏情况，所有主存块都被映射到同一个 Cache 行中，Cache 也能处理。主存-外存层次交换运行速度相比前者要慢很多（为毫秒级别）且交换数据量相比前者要大很多（交换单位为页，典型的页大小为 4 KB），如果采用直接映射方式，考虑最坏情况，所有页都被映射到同一个页框中，会出现频繁的调页，主存无法处理，所以主存-外存层次通常采用全相联映射方式。现代计算机一般采用虚拟存储技术，常见的虚拟存储映射方法包括：分页式存储管理、分段式存储管理、段页式存储管理

17 A

页式虚拟存储管理方式、组相联映射的 TLB 地址结构为：标记 | 组号 | 页内偏移量

虚拟地址为 32 位，则 TLB 地址长度为 32 位

页大小为 1KB，则页内偏移量为 10 位

TLB 共 32 个表项，采用 4 路组相联，则共有 $32/4 = 8$ 组，组号为 3 位

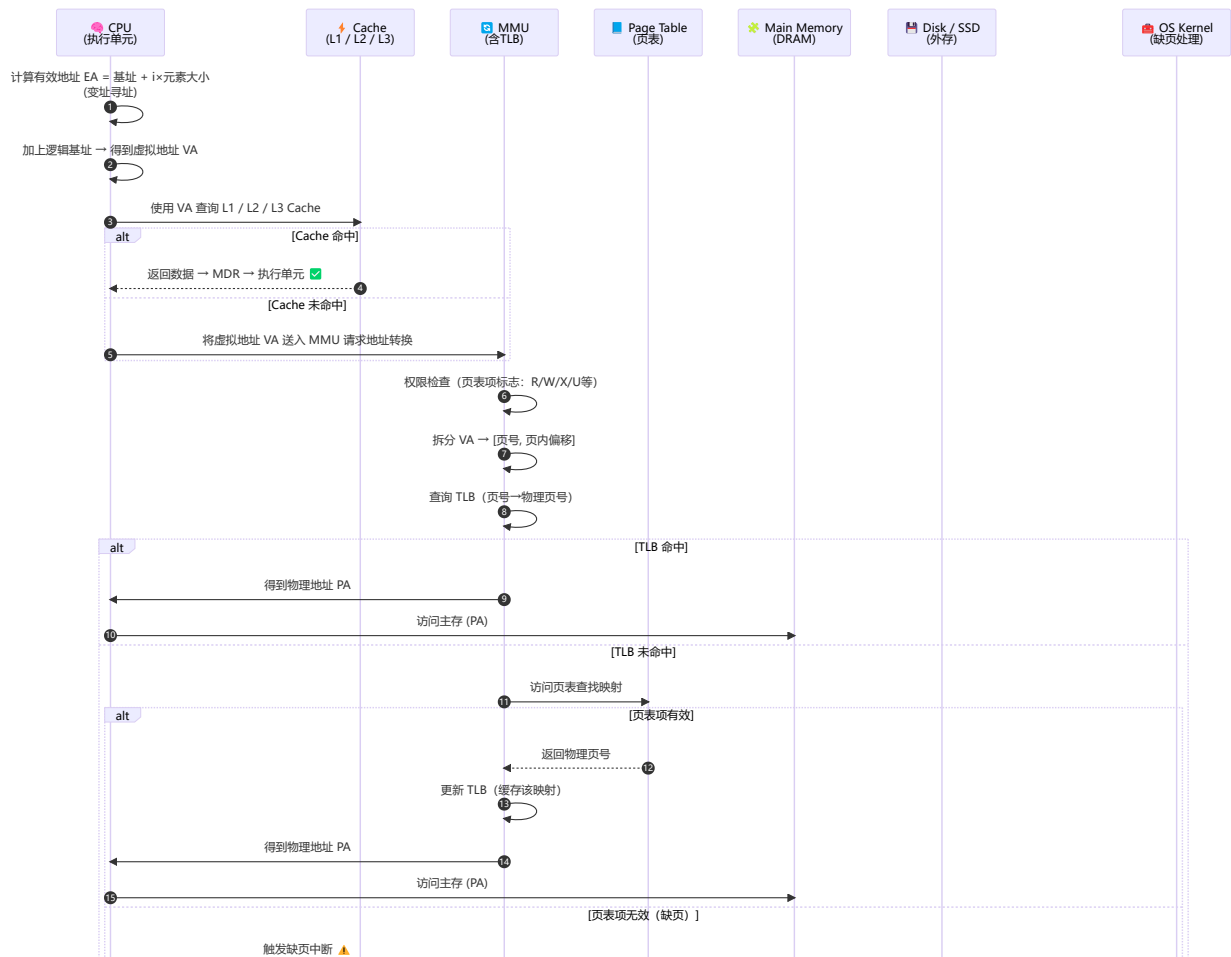
于是标记至少 $32 - 10 - 3 = 19$ 位

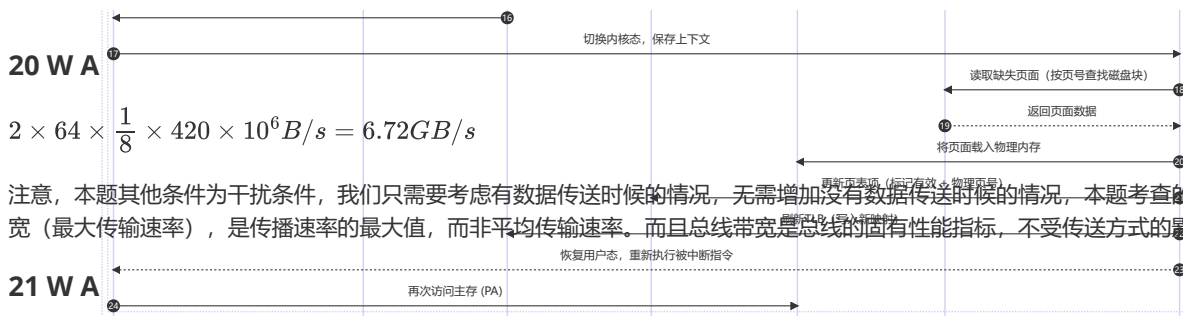
18 W M

MMU 与 Cache 缺失无关，不会检查 Cache 缺失

内存访问全局流程 // TODO 传给 Cache 是物理地址还是逻辑地址？这个图有问题

以 `load arr[i]` 为例，涉及计组/OS 交叉领域





$$2 \times 64 \times \frac{1}{8} \times 420 \times 10^6 B/s = 6.72 GB/s$$

注意，本题其他条件为干扰条件，我们只需要考虑有数据传送时候的情况，无需增加没有数据传送时候的情况，本题考查的是带宽（最大传输速率），是传播速率的最大值，而非平均传输速率。而且总线带宽是总线的固有性能指标，不受传送方式的影响

- 选项 A：中断优先级包含响应优先级和处理优先级，
 - 响应优先级是指 CPU 响应各中断源请求的优先次序，这种优先级往往是硬件电路已经设置好的，不便于改动
 - 处理优先级是指 CPU 实际对各中断源请求的处理优先次序。若不采用屏蔽技术，则应的优先次序就是处理的优先次序。若采用屏蔽技术，则中断屏蔽字用于确定中断处理的优先级。因此选项 A 错误
- 选项 B：中断过程主要包括中断响应和中断处理两个阶段。中断响应阶段包含保存断点和程序状态字。因此选项 B 正确
- 选项 C：中断过程主要包括中断响应和中断处理两个阶段。
 - 中断处理过程包含保存通用寄存器，中断处理过程由中断服务程序进行处理
 - 逻辑顺序为“先写 Cache 再送 MDR”，硬件实现中二者几乎同步（流水并行）
 - 在中断服务程序中设置适当的屏蔽字，能起到对优先级不同的中断源的屏蔽作用
- 选项 D：区别于多重中断方式，单重中断方式不允许中断嵌套，中断处理时 CPU 处于关中断（即禁止 CPU 响应中断源的中断请求）状态。因此选项 D 正确

24 W

对题干“不一定执行”理解有误，理解成了不会执行

终止子孙进程的前提是存在子孙进程，故不一定执行

27 A

伙伴算法即伙伴分配算法 (Buddy Allocation Algorithm) 将可用内存分割成大小相等、成对的“伙伴”块。当请求内存时，系统会找到大于等于请求大小的最小块 (2^i ，若不存在则继续向上找 $2^{i+1} \dots$)，并分配给请求的大小，将剩余部分形成新的伙伴块。这种方法有利于减少外部碎片，但可能会生成一定量的内部碎片

- 内部碎片：已经被分配出去但未被完全利用的内存空间
- 外部碎片：未被分配的内存空间，但由于这些空间过小或不连续，无法满足新的内存分配请求

28 A

线程可共享其所属进程的地址空间和资源，包括堆和全局变量。每个线程都有自己的栈，用于保存其局部变量和返回地址

29 A

- open() 系统调用用于打开文件，是通过文件按名查找实现的，open() 系统调用返回一个文件描述符，该描述符用于后续的文件操作，如读取、写入等
- read() 系统调用用于从文件中读取数据。read() 需要在文件已被打开的情况下才能实现
- write() 系统调用用于向文件中写入数据。write() 需要在文件已被打开的情况下才能实现
- close() 系统调用用于关闭一个打开的文件。close() 需要在文件已被打开的情况下才能实现

30 A

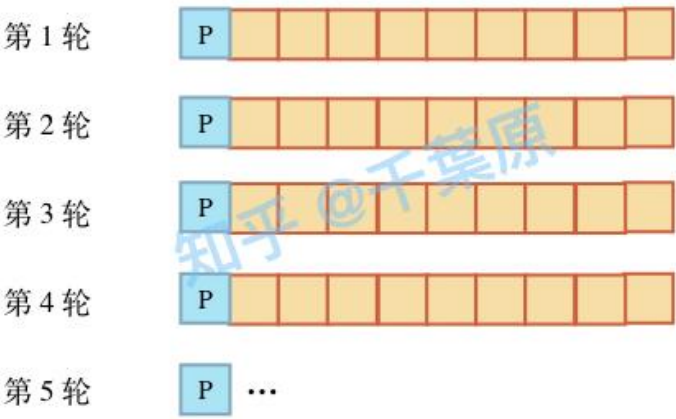
举一反三

假设某系统使用时间片轮转调度算法进行 CPU 调度，时间片大小为 5 ms，系统共有 10 个进程，初始时均处于就绪队列，执行结束前仅处于执行态或就绪态。若队首的进程 P 所需 CPU 时间最短，时间为 25 ms。在不考虑系统开销的情况下，则进程 P 的周转时间为

- A. 200 ms
- B. 205 ms
- C. 250 ms

D. 295 ms

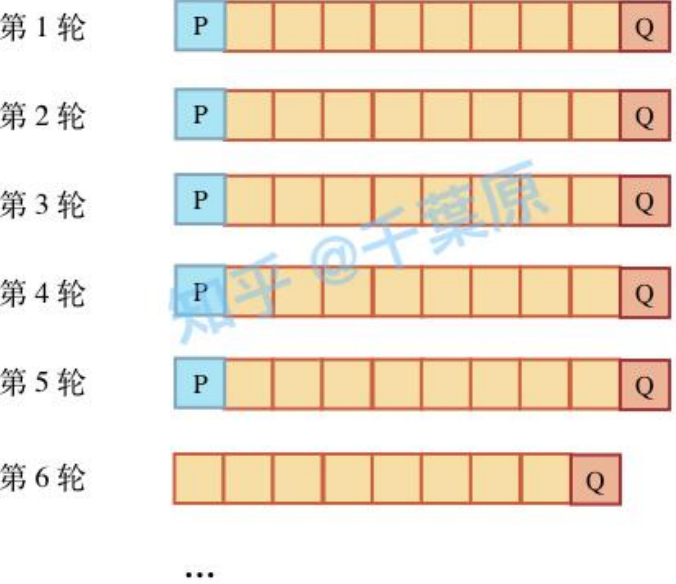
参考答案：B，示意图如下。



假设某系统使用时间片轮转调度算法进行 CPU 调度，时间片大小为 5 ms，系统共有 10 个进程，初始时均处于就绪队列，执行结束前仅处于执行态或就绪态。若队首的进程 P 所需 CPU 时间最短，时间为 25 ms，队尾的进程 Q 所需 CPU 时间第二短，时间为 30 ms，在不考虑系统开销的情况下，则进程 Q 的周转时间为

- A. 200 ms
- B. 205 ms
- C. 250 ms
- D. 295 ms

参考答案：D，示意图如下。

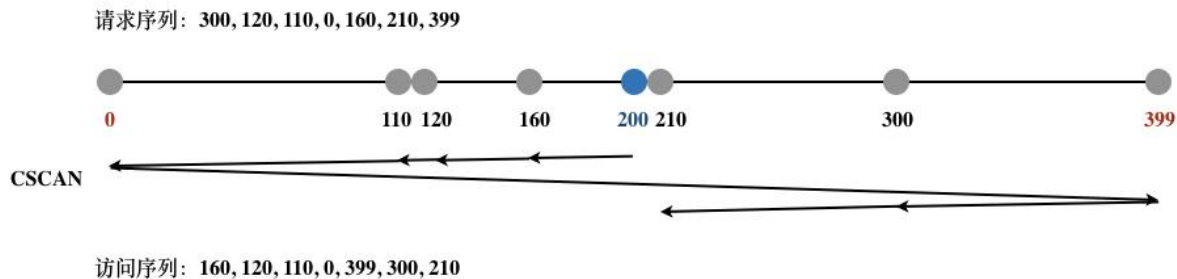


31 A

当键盘输入数据时，首先会将数据发送到键盘控制器的数据寄存器中。当键盘控制器的数据寄存器接收到数据后，它会触发一个中断请求 (IRQ1)，通知 CPU 有数据等待处理。CPU 响应中断请求后，会执行键盘中断服务例程。在中断服务例程中，CPU 从键盘控制器的数据寄存器读取输入数据，并将其存放到 **内核缓冲区** 中

32 A

$$|0 - 200| + |0 - 399| + |210 - 399| = 788$$



34 A M

- ASK (Amplitude Shift Keying) 幅移键控, 这种调制方式是利用载波幅度的变化来传送数字信息的
- PSK (Phase Shift Keying) 相移键控, 这种调制方式是利用载波振荡相位的变化来传送数字信息的
- FSK (Frequency Shift Keying) 频移键控, 这种调制方式是利用载波频率的变化来传送数字信息的。在二进制数字调制中, 常见的是二进制 0 和 1 对应不同的频率
- DPSK (Differential Phase Shift Keying) 差分相移键控, 它不是利用载波相位的绝对数值传送数字信息, 而是用前后码元的相对载波相位值传送数字信息

36 W A

冷门协议 CSMA/CA

$$DATA = \frac{1998B}{54Mb/s} = \frac{1998 \times 8b}{54 \times 10^6 b/s} = 296\mu s$$

$$NAV = SIFS + DATA + SIFS + ACK = 28 + 296 + 28 + 2 = 354\mu s$$

802.11 预约信道机制 CSMA/CA 协议

- DIFS: 信道空闲后, 发送数据帧前必须等待的一段固定时间, 用于确保信道真正空闲、避免冲突
- SIFS: 最短时间间隔
- NAV: 网络分配向量, 指出信道忙的持续时间, 除发送站和接收站外的站点都不能在这段时间内发送数据

RTS (Request To Send) 和 CTS 帧 (Clear To Send)

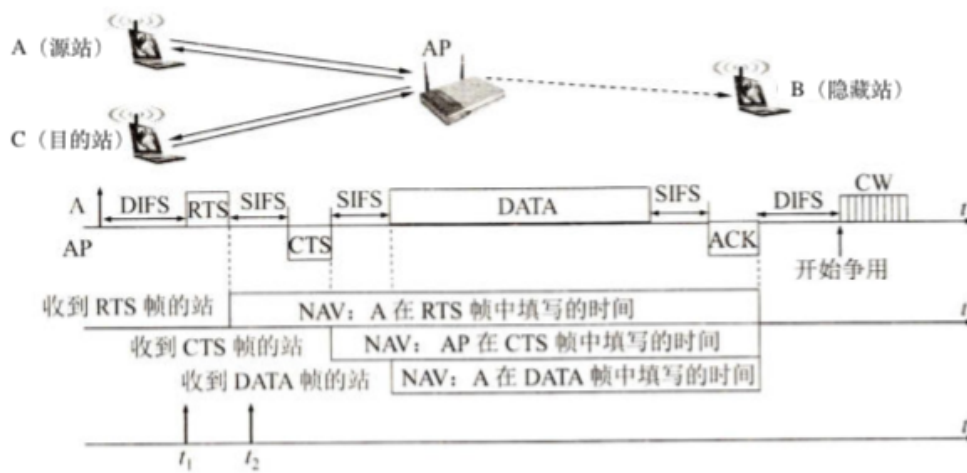
- RTS 帧: 站点 A 向接入点 AP 预约信道的帧, 首部包含预约时间
 $SIFS + CTS + SIFS + DATA + SIFS + ACK$
- CTS 帧: AP 广播响应, 停止其他隐藏站的数据发送, 首部包含预约时间 $SIFS + DATA + SIFS + ACK$

源站 (站点 A) 发送数据帧

在源站发送数据帧首部中, 有时间 $DATA + SIFS + ACK$, 这个时间的用途是: 如果有的站没有收到 RTS 和 CTS 帧, 那么收到数据帧后, 也能设置其 NAV

总结

- 收到源站发出的 RTS 帧的站设置其 NAV = $[SIFS + CTS + SIFS + DATA + SIFS + ACK]$
- 收到接入点 AP 发出的 CTS 帧的站设置其 NAV = $[SIFS + DATA + SIFS + ACK]$
- 没有收到以上 RTS 和 CTS 帧但收到数据帧的站设置其 NAV = $[DATA + SIFS + ACK]$

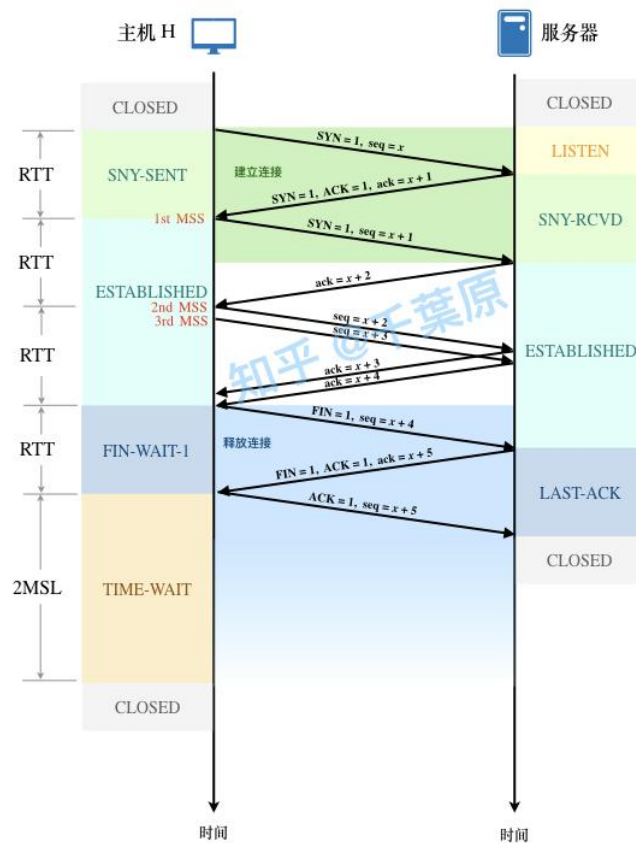


发送 RTS 帧和 CTS 帧对信道进行预约

38 A

需要考虑慢开始，两个 RTT 内数据就发送完了

注：三次握手最后一个报文可以携带数据（连接已建立）；四次挥手第一个报文不能携带数据（表示数据发送完毕）



39 W A

错在没有对结果取反

```

1  1011 1001 1011 0110
2  + 0110 0101 1100 0101
3  -----
4  0001 1111 0111 1011
5  +                               1 // 高位进位
6  -----
7  0001 1111 0111 1011
8  1110 0000 1000 0100 // 加完一定记得取反！

```

通用校验和计算规则 **一补校验**（One's Complement Checksum）（适用于 IP/TCP/UDP 等）

- 1. 将 UDP 数据报的伪头部（包含源 IP 地址、目的 IP 地址、协议类型和 UDP 长度）、UDP 头部和数据拼接成一个连续的二进制流
- 2. 将这个二进制流视为一系列 16 位整数，对这些整数进行累加。在累加过程中，如果发生溢出（即超过 16 位），将高位的进位加回到低 16 位，这一过程可能需要多次循环，直到没有溢出
- 3. 取累加后的低 16 位，然后对其进行 **按位取反**（即求反码），得到的就是校验和

协议层	协议	校验内容范围	是否包含伪首部	校验算法	说明
网络层	IPv4	IP 首部（不含数据部分）	✗ 否	16 位一补和	仅校验首部正确性，每次转发都要重新计算
网络层	IPv6	无（弃用）	—	—	IPv6 不再使用首部校验和，依赖上层（TCP/UDP）校验
传输层	UDP	UDP 首部 + 数据 + 伪首部	✓ 是	16 位一补和	IPv4 中可选、IPv6 中强制
传输层	TCP	TCP 首部 + 数据 + 伪首部	✓ 是	16 位一补和	用于端到端数据可靠传输
链路层	以太网帧（FCS）	整个 MAC 帧（除帧校验序列本身）	✗ 否	CRC32（循环冗余校验）	与上层的“校验和”不同，是多项式除法结果
链路层	PPP、HDLC 等	帧数据部分	✗ 否	CRC16/CRC32	同样使用多项式校验

40 A

协议版本	HTTP/1.0	HTTP/1.1	HTTP/2.0	HTTP/3.0
连接方式	TCP（短连接）	TCP（长连接）	TCP（长连接）	QUIC（基于 UDP）
是否复用连接	否	是	是	是
并发请求	无	支持管线化	多路复用（帧级）	多路复用（流级）
队头阻塞	TCP 队头阻塞	TCP 队头阻塞仍存在	TCP 队头阻塞仍可能存在	无
握手携带请求	否	是（可在最后一次握 手报文中发送第一个 请求）	是	是
请求 HTML + n 静态资源最少 RTT	2 × (n+1)	2	2	1（0-RTT 可更低）
说明	每个资源都独立建立 TCP 连接，握手中可携带请求可优化 1 RTT，但仍为短连接模式	长连接 + 管线化，HTML + n 个资源共享 TCP 连接，仍可能受队头阻塞影响	所有请求在同一连接并行发送，HTTP 层消除队头阻塞，TCP 层仍有可能阻塞	QUIC 内建加密，多路复用，消除 TCP 队头阻塞，首次连接可 1 RTT 或 0-RTT

大题

41

需要优化算法描述的表述

构造一个可以容纳 $G.numVertices$ 个元素的数组作为辅助队列。初始时，所有入度为 0 的顶点都被放入队列中，检查队列中元素个数，若队列中元素个数大于 1，则意味着拓扑序列不唯一，返回 0。在广度优先搜索的每一步中，我们取出队首的结点 u ，我们移除 u 的所有出边，也就是将 u 的所有相邻顶点 v 的入度减少 1，如果某个相邻顶点 v 的入度变为 0，那么我们就将 v 放入队列中，若队列中元素个数大于 1，则意味着拓扑序列不唯一，返回 0。重复该过程，直到队列为空。检查顶点入度表，若所有顶点的入度均为 0，则意味着存在唯一的拓扑序列，返回 1，否则说明 G 中存在环，意味着不存在拓扑排序，返回 0

[\(10 封私信\) 2024 年 408 真题数据结构篇 - 知乎](#)

42

// TODO

43

// TODO

44

// TODO

45

// TODO

46

第二问：注意这里只执行一次！题目简化为简单的同步问题，一个信号量即可控制二者执行的先后顺序。不是生产者-消费者模型，生产者-消费者模型是在一个循环里不断执行的

47

冷门协议

// TODO

2025 年

选择题

5 W

唉真题少印个 8，cnmd，这两分不算

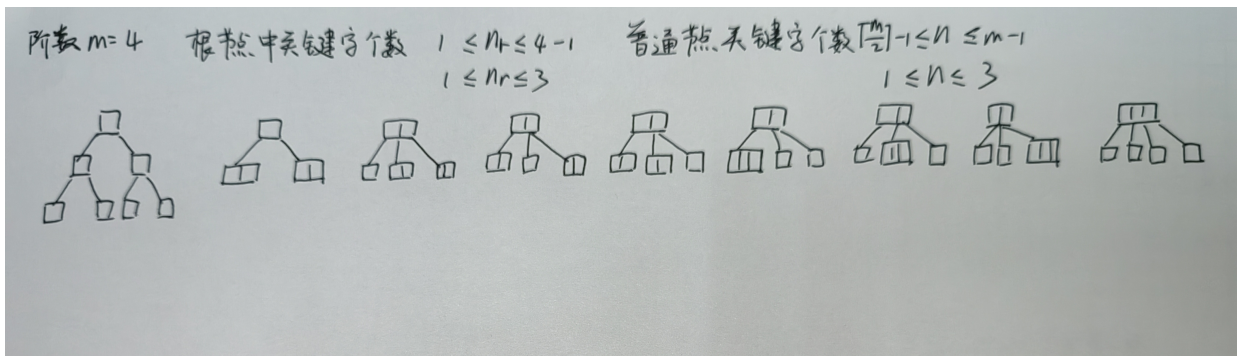
7 A M

分块大小设置为 $\sqrt{n}$ 时，分块查找的平均查找长度最小

8 W A

B 树阶数为 m ，总关键字个数为 N ，根节点中关键字个数为 n_r ，普通节点中关键字个数 n ，树高为 h （从 1 开始）

1. 对每个节点：分支数（孩子数）= 关键字个数 + 1
2. 令 $t = \lceil \frac{m}{2} \rceil$
3. $1 \leq n_r \leq m - 1$
4. $t - 1 \leq n \leq m - 1$
5. $2t^{h-1} - 1 \leq N \leq m^h - 1$
6. $\lceil \log_m(n + 1) \rceil \leq h \leq \lfloor \log_t(\frac{n+1}{2}) \rfloor + 1$



10 W

排序算法	最好情况比较	最坏情况比较	平均比较	最好移动	最坏移动	平均移动
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(0)$	$O(n^2)$	$O(n^2)$
插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(n)$	$O(n^2)$	$O(n^2)$
快速排序	$O(n \log n)$	$O(n^2)$	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(n \log n)$
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n)$	$O(n)$	$O(n)$

12 W S

没有注意到赋值后符号位会扩展

1. $SI = -32767$ ，其二进制补码(16 位)是 1000000000000001
2. 当把 SI 赋值给 int 时，进行符号位扩展到 32 位得到 1111111111111111100000000000001
3. 这个值在无符号整数表示下是 $2^{32} - 2^{15} + 1$

二刷

又没有注意到赋值后符号位会扩展

通用计算模型

位宽 = N ，有符号数值 = x

$$U = 2^N + x$$

题型	已知条件	要求	解法公式/方法	示例
十进制有符号数 $\rightarrow$ 补码	一个整数 x ，位宽 N	写出 N 位补码	$x \geq 0 \Rightarrow$ 补码 = x $x < 0 \Rightarrow$ 补码 = $2^N + x$	-5 的 8 位补码： $2^8 - 5 = 251 = 11111011$
补码 $\rightarrow$ 真值 (有符号数)	N 位补码 (二进制串)	对应的十进制整数	若最高位 = 0: 正数，直接转十进制 若最高位 = 1: 负数，结果 = $X - 2^N$	11111011 (8 位) $\rightarrow 251 - 256 = -5$
补码负数 $\rightarrow$ 无符号数	有符号值 $x < 0$ ，位宽 N	对应无符号数值	$U = 2^N + x = 2^N - x $	-32767 的 32 位无符号： $2^{32} - 32767$
无符号数 $\rightarrow$ 有符号数	N 位二进制串	对应的十进制有符号数	若 $U < 2^{N-1}$ ，真值 = U 若 $U \geq 2^{N-1}$ ，真值 = $U - 2^N$	32 位无符号 4294967295 $\rightarrow$ 真值 = -1
符号扩展 (有符号数)	N 位补码	扩展为 M 位补码 ($M > N$)	复制符号位到高位	8 位 11111011 (-5) $\rightarrow$ 16 位： 11111111 11111011
移位运算	补码或无符号数	移位结果	逻辑移位: 左/右补 0 算术右移: 补符号位	-8 的 8 位补码: 11111000 算术右移 1 $\rightarrow$ 11111100 (-4)

题型	已知条件	要求	解法公式/方法	示例
数值范围	位宽 N	有符号/无符号范围	有符号: $[-2^{N-1}, 2^{N-1} - 1]$ 无符号: $[0, 2^N - 1]$	16 位: 有符号范围 $[-32768, 32767]$, 无符号范围 $[0, 65535]$

14 A

本题用双符号位可以判断溢出

16 A M

- 选项 A: 是否采用阵列乘法器, 这属于 **处理器内部** 的实现细节, 不属于 ISA 规定的内容。阵列乘法器是一种硬件实现乘法运算的电路, 它与 ISA 没有直接关系。
- 选项 B: 是否采用定长指令字格式, 这是由 **ISA** 规定的。ISA 会规定指令的格式, 包括指令长度是固定的还是可变的等情况。
- 选项 C: 是否采用微程序控制器, 这是 **处理器内部** 的控制机制, 属于硬件实现层面, 不属于 ISA 规定的内容。微程序控制器是用于产生控制信号的一种硬件结构。
- 选项 D: 是否采用单总线数据通路, 这也是 **处理器内部** 的数据传输机制, 属于硬件设计细节, 不属于 ISA 规定的内容。单总线数据通路是一种硬件架构。

18 A M

Cache 缺失率增加时, CPI 也会增大

有关单周期 CPU 和流水线 CPU

对比维度	单周期 CPU	多周期 CPU	流水线 CPU
本质区别	所有指令在 一个时钟周期 内完成执行	每条指令拆分为多个步骤, 在 多个时钟周期 内完成	指令分段并行执行, 每条指令仍需多周期, 但多条指令可重叠执行
部件冗余	高冗余 : 数据通路按最慢指令设计, 部件利用率低	低冗余 : 部件分时复用 (如 ALU), 硬件利用率高	部件复用 + 并行执行, 提高利用率
时间利用率	低效 : 时钟周期 = 最慢指令时间, 简单指令等待时间长	高效 : 时钟周期按最短步骤设计, 空闲部件可执行其他指令	时钟周期 = 最慢流水段时间, 短阶段时间浪费小
CPI	固定 CPI = 1 (每条指令 1 周期)	可变 CPI > 1 (不同指令周期数不同, 如加法 = 4 周期, LOAD = 5 周期)	理想情况下 $CPI \approx 1$ (考虑冒险后略大于 1)
时钟频率	低频率 : 由最慢指令 (如 LOAD) 决定	高频率 : 由最长执行阶段 (如访存) 决定	高频率 : 由最慢流水段决定
控制逻辑	简单组合逻辑, 直接生成控制信号	复杂状态机控制, 需多周期时序协调	控制逻辑需处理流水线阶段、冒险与冲突
硬件复杂度	数据通路复杂 (宽而深), 控制简单	数据通路精简 (窄而浅), 控制逻辑复杂	数据通路介于两者, 控制逻辑复杂 (冒险处理)
性能瓶颈	受限于最慢指令, 无法优化局部操作	可通过优化关键阶段 (如 ALU) 提升整体性能	受最长流水段影响, 阶段不平衡会限制时钟频率
典型应用场景	教学模型 (简单直观)	实际处理器基础 (如 MIPS 多周期实现)	高性能 CPU、现代处理器
功耗	较高 (每个周期激活全部部件)	较低 (仅激活当前阶段所需部件)	介于单周期与多周期之间, 根据流水线深度不同
时钟周期确定原则	最慢指令时间	最长步骤时间	最长流水段时间

- 单周期 CPU → “最慢指令说了算”

多周期 CPU → “最长步骤说了算”

流水线 CPU → “最长流水段说了算”

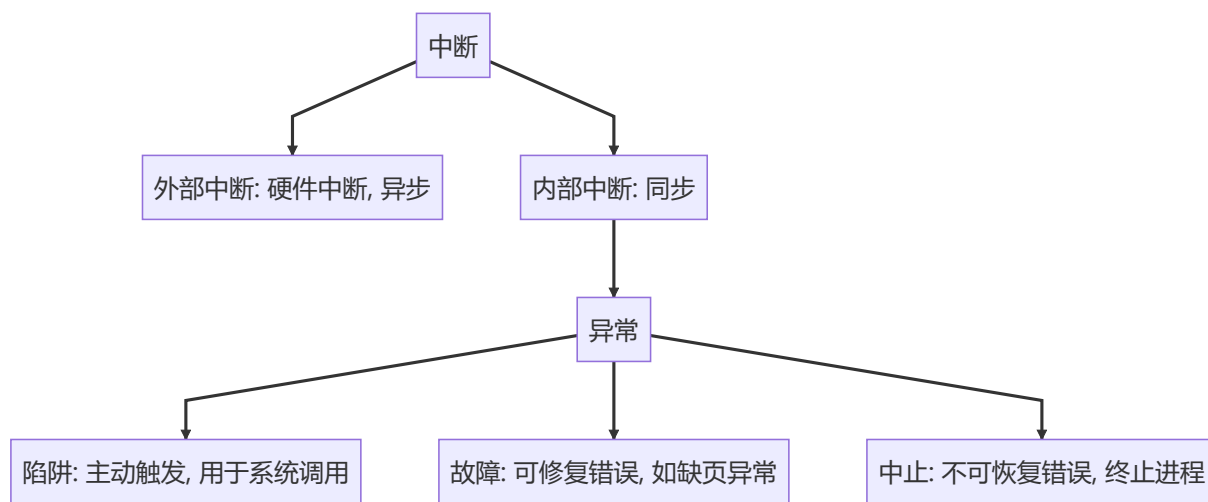
22 A S

外部中断：外部中断是由 **外部设备向 CPU** 发出的中断请求信号引起的中断

- A: DMA（直接内存访问）控制器在完成数据传送后，会向 CPU 发出一个中断请求，告知 CPU 数据传送已经完成。这个中断请求是由外部设备（DMA 控制器）发出的，属于外部中断
- B: 总线事务结束通常是由 CPU 内部的总线控制逻辑处理的，它不会产生外部中断请求
- C: 页故障处理是由内存管理单元（MMU）和操作系统共同处理的，属于内部中断的范畴，不会产生外部中断请求
- D: 执行断点指令会引发 **调试中断**，这是由 CPU 内部的调试机制产生的中断，不属于外部中断

二刷

做到假卷子了，这题是对的



24 A S

VMM（虚拟机监控器）的特权级高于操作系统，负责管理和分配硬件资源给虚拟机和其中的操作系统，需要对硬件有完全的控制能力

二刷

通过虚拟机技术，可以用一台主机上模拟多种 ISA（指令集架构）

26 A

LRU 是 Least Recently Used 的缩写，即最近最少使用

27 W A

注意题目中是 **进程运行** 所需的最小页框数

指令系统支持的寻址方式：指令系统的寻址方式对确定进程运行所需的最少页框数有重要作用。不同的寻址方式决定了进程能够访问的内存范围和方式。例如，如果指令系统支持间接寻址，并且可以通过多级间接寻址访问大量的虚拟地址，那么为了保证进程能够正确运行，就需要提供足够的页框来满足这种寻址需求。如果没有足够的页框，可能会导致无法正确寻址，进而影响进程的运行。所以在确定进程运行所需的最少页框数时，需要考虑指令系统支持的寻址方式

28 A M

虚拟文件系统（VFS）：虚拟文件系统是一种 **抽象层**，它位于操作系统内核和具体的文件系统（如 ext4、NTFS 等）之间。它的主要目的是为不同的文件系统提供一个 **统一的接口**，使得操作系统能够以一种统一的方式处理各种不同类型的文件系统

29 W A M S

索引节点 (inode) 方式的文件系统：在这种文件系统中，**索引节点** 用于存储文件的元数据，如文件大小、文件的物理块位置、访问权限等信息。**目录文件** 主要包含文件名和对应的索引节点号，用于通过文件名查找文件的索引节点

二刷

在目录文件中写入 F 的访问权限信息：文件的访问权限信息（如读、写、执行权限）存储在文件 F 的 **索引节点** 中，而不是目录文件中。目录文件仅存储文件名和索引节点号的映射关系。因此，选项 C 是文件系统不会做的操作

30 A S

内存映射文件：将文件的全部或部分内容映射到进程的 **虚拟地址空间**，进程可以像访问普通内存一样操作文件内容
访问操作系统会自动处理：

- 虚拟地址 → 物理地址的转换（页表）
- 文件映射 **虚拟地址**，物理内存按需加载，缺页时将磁盘块内容加载到物理内存
- 文件内容更新同步到磁盘（取决于映射类型，如 MAP_SHARED）

主要特性与作用：

特性	作用与说明	示例 / 应用场景
进程间通信 (IPC)	多个进程可以映射同一个文件，实现数据共享与通信	数据库系统多个进程共享访问数据库文件，修改即时可见
虚拟地址映射	文件映射到进程虚拟地址空间，而不是直接映射到物理内存	通过 <code>mmap</code> 系统调用将文件内容映射到指定虚拟地址，访问方式如操作内存
页面到磁盘块映射	当访问映射区域页面缺失时，操作系统根据文件的磁盘块分配，将内容加载到内存	虚拟页面缺页时，OS 根据 inode 的磁盘块指针加载对应数据到物理内存

二刷

如何理解页面到磁盘块的映射？

调用 `mmap` 并不会加载数据，OS 只在进程页表中创建 **虚拟页面条目**，并标记这些页为“未加载”。首次访问时 CPU 发现虚拟页面不在内存中触发缺页异常，此时 OS 介入，通过 inode 找到磁盘块，加载 **数据** 到物理内存，随后更新页表
这一过程可以理解为页面到磁盘块的映射：OS 在背后透明地处理虚拟内存页面与磁盘文件块之间的映射关系，让程序员可以像访问普通内存一样访问文件内容

32 W A M S

温彻斯特硬盘：传统机械硬盘
无论是机械硬盘还是固态硬盘，文件系统都需要确定盘块大小

二刷

文件系统与硬盘之间还有一层硬盘驱动器（无论是机械还是固态），其职能包括但不限于划分扇区、降低寻道时间、实现均衡磨损

33 W A S

[\(9 封私信\) 哈工大计算机|必考知识点解读——第一章计算题（上） - 知乎](#)
[\(10 封私信\) 哈工大计算机|必考知识点解读——第一章计算题（下） - 知乎](#)

二刷

$$T_{CS} = 32 \mu s + \frac{2 MB}{10 Mb/s} = 32 \mu s + 1.6 s$$

$$T_{MS} = \frac{2 MB}{10 Mb/s} + \frac{2 MB}{100 Mb/s} + \frac{2 MB}{1000 Mb/s} = 1.6 s + 0.16 s + 0.016 s$$

$$T_{PS} = \frac{2 MB}{400 B} \cdot \frac{400 B}{10 Mb/s} + \frac{400 B}{100 Mb/s} + \frac{400 B}{1000 Mb/s} = 1.6 s + 32 \mu s + 3.2 \mu s$$

34 W A N S

基于最小汉明距离的差错控制理论

汉明距离 (Hamming Distance)：指两个等长码字之间对应位不同的数量。编码集的 **最小汉明距离** (d_{min}) 是所有码字对之间汉明距离的最小值，它决定了编码的检错和纠错能力

检错能力：可以检测出最多 e 位错误，其中 $e \leq d_{min} - 1$

纠错能力：可以纠正最多 t 位错误，其中 $t \leq \lfloor (d_{min} - 1)/2 \rfloor$

标准汉明码的 $d_{min} = 3$ ，可以纠正单比特错误或检测双比特错误

二刷

1. 为了检测 e 个位错，海明距离 $d \geq e + 1$ 。这样能够保证一个有效码字在发生小于等于 e 个位错时，一定不会变成另一个有效码字
2. 为了纠正 t 个位错，海明距离 d 至少为 $2t + 1$ ，于是 $t \leq \lfloor \frac{d-1}{2} \rfloor$

35 W N M S

二进制指数退避算法

退避时间 = 随机数 $\times$ 时隙时间

随机数 $\in [0, 2^{\min(k, 10)} - 1]$

(隐含条件) IEEE 802.3 规定：时隙时间 = $512 \times$ 比特时间 = $512/\text{数据速率}(Mbps)$

38 A

注意：cwnd 增加都发生在确认报文后

MSS 决定了每个 TCP 段能够携带的最大数据量，建议将题目中的拥塞窗口、发送窗口和拥塞控制阈值转换为 MSS 的倍数

题目中甲之前发送的数据量是 $1MSS$ ，接受到一个 ACK 显示乙的接收窗口为 $4MSS$ ，在甲未收到新的确认段前，根据慢开始，下一轮甲的拥塞窗口倍增至 $2MSS$ ，即可以发送两个 TCP 段

40 A

答案错了，我的选项没错

POP3 主要用于用户从邮件服务器上把邮件下载到本地计算机，不能发送

协议名称	主要功能	能否发送邮件?	能否接收邮件?	工作方式/特点
SMTP	发送邮件	能	不能	用于将邮件从客户端 推送 到邮件服务器，或在不同邮件服务器之间 中继 邮件。是“只发不收”的协议
POP3	下载邮件	不能	能（下载并通常删除）	用于将邮件从服务器 下载 到本地计算机。通常是“下载后删除”模式，邮件在本地管理
IMAP	管理邮件	能（间接）	能（在线管理）	用于在服务器上 直接管理和同步 邮件。所有操作（读、删、移动）都会在服务器同步，允许从多个设备访问同一邮箱状态。 发送邮件本身仍由 SMTP 处理 ，但客户端通过 IMAP 管理“已发送”文件夹

大题

41

```
1  /**
2   * @see 2025年 41题
3   * 时间复杂度 O(n)
4   * 空间复杂度 O(n)
5   */
6  void calc_mul_max(int a[], int res[], int n) {
7      int* suff_max = new int[n + 1]; // 后缀最大值数组
8      int* suff_min = new int[n + 1]; // 后缀最小值数组
9
10     for (int i = 0; i <= n; ++i) {
11         suff_max[i] = INT_MIN;
12         suff_min[i] = INT_MAX;
13     }
14
15     for (int i = n - 1; i >= 0; --i) {
16         suff_max[i] = std::max(suff_max[i + 1], a[i]);
17         suff_min[i] = std::min(suff_min[i + 1], a[i]);
18     }
19
20     for (int i = 0; i < n; ++i) {
21         res[i] = a[i] * (a[i] >= 0 ? suff_max[i] : suff_min[i]);
22     }
23
24     delete[] suff_min;
25     delete[] suff_max;
26 }
```

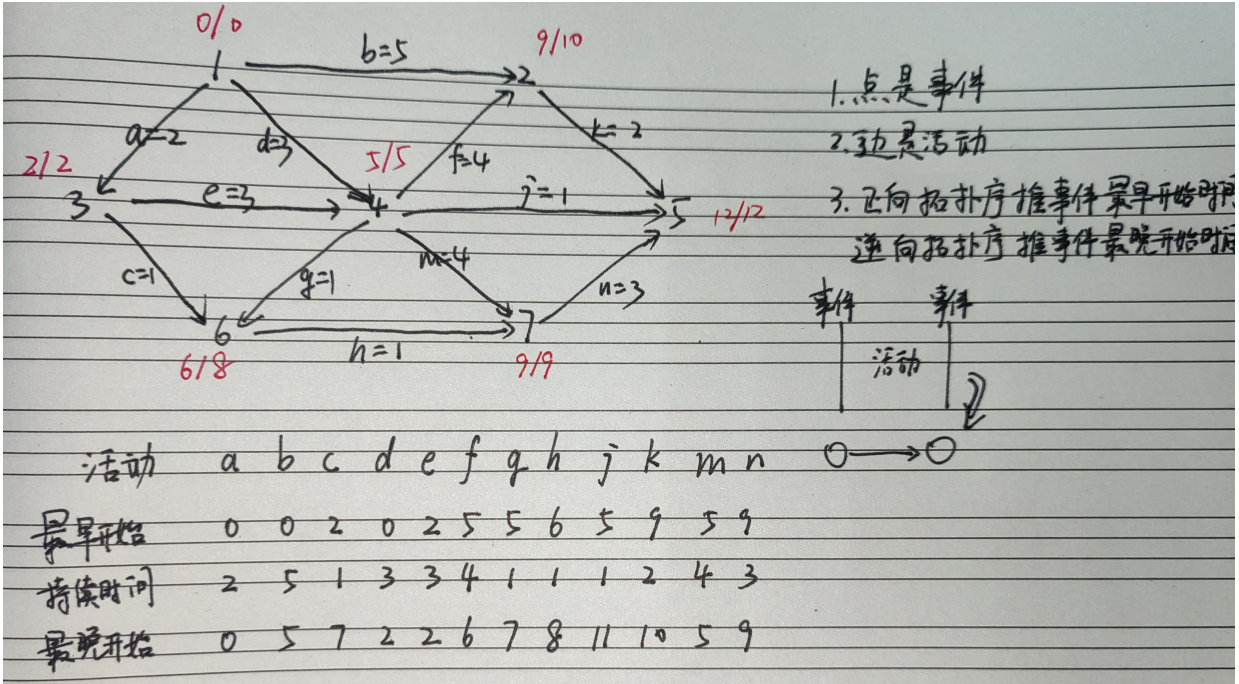
二刷

```
1 void calc_mul_max(int a[], int res[], int n) {
2     int suff_max = INT_MIN, suff_min = INT_MAX; // 用两个变量压缩后缀数组
3     for(int i = n - 1; i >= 0; --i) {
4         suff_max = std::max(suff_max, a[i]);
5         suff_min = std::min(suff_min, a[i]);
6         res[i] = a[i] * (a[i] >= 0 ? suff_max : suff_min);
7     }
8 }
```

42

b 的持续时间为 5，最晚开始时间为 5，若 b 延期到 6 开始，则 b 的持续时间只能被压缩为 4

若不压缩 b，需要重新构建 AOE 网络的最早开始时间和最晚开始时间，找到新的关键路径，然后再压缩关键路径上的活动保证总用时与计划一致



43

做 408 题一定要注意细节!!!

首先看到代码 `d[i] = d[i] / x`, 马上想到涉及两次访存

关于 Cache 索引

- 直接映射: 行号
- 全相联映射: 没有索引
- 组相联映射: 组号

关于十六进制数组首地址算偏移量, 本题是按字节编址, 直接偏移 100 个 4 字节即可, $01800120H + 100 * 4 = 018001B0H$, 注意偏移量是十进制数, 组号在这个结果上取第 6 位到第 11 位

这道题关键在于数组首元并不存放在主存块开头, 地址偏移了 20H, 即首元所在的主存块共 64B, 前 32B 没存数组, 导致数组占用的主存块个数比一般情况 $\frac{2048 \times 4B}{64B} = 128$ 个块多 1, 即 129 个块。此外在页中, 同理, 占用的页数为 $2 + 1 = 3$ 个, 于是缺页也是 3 次

于是 4096 次访存有 129 次 miss, 即可算出 Cache 缺失率

注意题目中说: Cache 命中时间为 2 个时钟周期, 缺失损失为 200 个时钟周期。意思是 hit 总用时 2 个时钟周期, miss 总用时 $200 + 2$ 个时钟周期

44

补码除法电路即便不记得也可以推

首先问寄存器初始化值, 除数肯定放在 Y, 对于被除数, 注意到题目中说 SEXT 表示按符号扩展, 于是被除数符号扩展为 `0xffffffff87654321` 分别放入 R 和 Q 中。计数器显然只有控制逻辑计数器有。对于补码除法, 实际上是在高 32 位 (部分余数) 基础上不断减去除数, 如果够减 (部分余数与除数符号相同) 就用部分余数减去除数, 若不够减 (部分余数与除数符号不同) 就用部分余数加上除数。于是需要用到的 ALU 运算为补码加法、补码减法

补码除法异常情况有两种, 一是除零异常, 二是当被除数为 INT_MIN (`0x80000000`), 除数为 -1 (`0x11111111`) 时, 二者相除结果会正溢出 (例如 $-127 / -1 = 128$ 正溢出)

在异常响应过程中, CPU 需要完成哪些操作? 一个标准回答

- 将断点和程序状态保存到内核栈或特定寄存器中, 关中断, 最后跳转到内核中的除法异常处理程序执行

补码乘法就是不断做加法和右移，高位用来存放积
补码除法就是不断做减法和左移，低位用来存放商

45

PV 函数首先要用 while(true) 包裹

二刷

```
1 semaphore mutex = 1; // 铁锹互斥量
2 semaphore cnt = 0; // 当前树坑个数
3 semaphore cnt2 = 3; // 当前还可挖的坑个数
4 semaphore cnt3 = 0; // 当前填了土的树坑个数
5
6 // 甲
7 P1() {
8     while(true) {
9         wait(cnt2);
10        wait(mutex);
11        挖坑;
12        signal(mutex);
13        signal(cnt);
14    }
15 }
16
17 // 乙
18 P2() {
19     while(true) {
20         wait(cnt);
21         放树苗;
22         wait(mutex);
23         填土;
24         signal(mutex);
25         signal(cnt3);
26     }
27 }
28
29 // 丙
30 P3() {
31     while(true) {
32         wait(cnt3);
33         浇水;
34         signal(cnt2);
35     }
36 }
```

三刷

甲乙丙构成生产者消费者的链条，看到生产者消费者不要固有思维设定当前树坑个数和还可以挖的树坑个数，而要根据实际情况增删信号量。这里就需要为乙和丙之间的同步关系增设一个当前填了土的坑数量的信号量

为了保证并发安全，最好等丙浇完水后才通知甲挖坑

46

二刷

- 注意：可读写数据区存全局变量和静态变量。用户栈存局部变量

解：linux系统下，进程执行时的地址空间如下所示：



下面按照虚拟地址递增顺序依次进行介绍。

（一）只读代码段

存储可执行代码的指令。只读，通常不可修改。

（二）数据段

存储全局变量和静态变量。

（三）运行时堆

由进程在运行时动态创建，该区域可以动态增长和收缩，例如 C 语言的 malloc() 可以申请该区域的内存，free() 可以释放该区域的内存。

（四）共享库

该区域用于存放公共的共享库代码和数据，例如 C 语言的标准库函数。

（五）用户栈

47

二刷

关于 GBN 老生常谈的问题，其信道利用率与 SR 协议计算方式一致， $U = \frac{W_T t_1}{t_1 + 2\tau + t_2}$ 。题目忽略 t_2 ，且

$t_1 = \frac{1500 \times 8b}{200 \times 10^3 b/s}$ ，于是可计算出 W_T 至少为 8。然后，关于帧序号位数 n ，三个协议均满足 $W_T + W_S \leq 2^n$ ，对于 GBN 协议 $W_S = 1$ ，于是 n 至少为 4

这道题有一个坑点：第三位的子网不能随便分，注意到管理区有一个主机的 IP 为 10.10.10.33/26，这就唯一确定了子网划分的答案！

杂项

进程就绪但未入就绪队列

进程处于就绪状态一定是在就绪队列中排队等待 CPU 调度。（错误）

原因：

1. 多核处理器中的进程迁移
 - 从繁忙核心迁移到空闲核心的就绪队列
2. 进程刚被唤醒，还未加入就绪队列
 - 进程从阻塞到就绪有两阶段，先变化状态到就绪，再加入就绪队列
3. 抢占结尾阶段正在切换上下文
 - 调度器决定抢占执行中的进程 P1，切换到 P2，P1 的状态被改为就绪，但内核仍需保存 P1 的上下文，保存完成后才将 P1 插入就绪队列

二级页表如何将虚拟地址转换为物理地址

- 虚拟地址由页号和页偏移组成
- 虚拟地址中的页号部分拆成两段，第一段 = 一级页表索引，第二段 = 二级页表索引
 - 虚拟地址中的页号部分被 **平均** 分为一级页号和二级页号（因为 **每一张页表的大小都是一页**）
- 一级页表（页目录）的 **物理基地址** 存储在专用寄存器中
- 用一级页号作为偏移量（考虑页目录项大小）查一级页表，一级页表的表项给出二级页表的 **物理基地址**（或页框号）
 - 若一级页表项无效，触发缺页或页表未建立
- 拿到二级页表物理基地址后，用二级页号作为 **偏移量**（考虑页目录项大小）查二级页表，二级页表项给出目标物理页框号
 - 若二级页表项无效，触发缺页
- 拼接物理页框号和页偏移得到物理地址

例题

假设某系统采用二级页表结构，虚拟地址 32 位，页大小 4KB，页表项占 4 字节。请回答：

- (1) 虚拟地址中页内偏移、一级页号、二级页号各占多少位？
- (2) 一级页表（页目录）的最大大小是多少？
- (3) 描述虚拟地址 0x00402010 到物理地址的转换过程
- (4) 如果在访问二级页表时发生缺页异常，可能的原因有哪些？

时间片轮转（RR）调度时间片长度对系统性能的影响

1. 时间片过长的影响，适合：**CPU-bound 程序多的批处理系统**
 - 表现为接近 **FCFS（先来先服务）调度**
 - 平均等待时间增大
 - **交互响应时间明显变差**（关键点）
 - 公平性下降：CPU 密集型进程长期占用处理器
 - 上下文切换开销减少（优点）
2. 时间片过短的影响，适合：需要快速交互响应的系统（如图形界面、Shell 等）
 - 系统变得“过度抢占”，调度频繁
 - **上下文切换开销显著增加**（关键点）
 - 有效 CPU 时间减少 → 吞吐量下降
 - 尽管响应快，但 **长作业进度推进缓慢**

扩展

1. 若时间片没有用完，进程就结束，那么 **立即调度就绪队列中的队首进程运行，并启动一个新的时间片**

例题

1. 时间片轮转调度算法适合的应用场景是（C）
 - A. 批处理系统中 CPU 密集型作业居多时
 - B. 实时系统中对截止时间要求严格时
 - C. 分时系统中需要保证交互响应时
 - D. 嵌入式系统中资源极度受限时
2. 某分时系统采用时间片轮转调度算法，现有 4 个进程 P1、P2、P3、P4 同时到达就绪队列，它们的估计运行时间分别为 12ms、5ms、8ms、4ms。请分析：
 - (1) 当时间片分别为 2ms 和 8ms 时，各自的平均周转时间和平均响应时间
 - (2) 从系统性能角度分析两种时间片设置的优缺点
 - (3) 如何确定合理的时间片长度，需要考虑哪些因素？

(1)

周转时间（完成时间 - 到达时间）：P1 = 29ms, P2 = 19ms, P3 = 25ms, P4 = 16ms

平均周转时间 = $(29 + 19 + 25 + 16) / 4 = 89 / 4 = 22.25\text{ms}$

响应时间（第一次获得 CPU 的时间）：P1 = 0ms, P2 = 2ms, P3 = 4ms, P4 = 6ms

平均响应时间 = $(0 + 2 + 4 + 6) / 4 = 12 / 4 = 3\text{ms}$

周转时间: $P1 = 29\text{ms}$, $P2 = 13\text{ms}$, $P3 = 21\text{ms}$, $P4 = 25\text{ms}$

平均周转时间 = $(29 + 13 + 21 + 25) / 4 = 88 / 4 = 22\text{ms}$

响应时间: $P1 = 0\text{ms}$, $P2 = 8\text{ms}$, $P3 = 13\text{ms}$, $P4 = 21\text{ms}$

平均响应时间 = $(0 + 8 + 13 + 21) / 4 = 42 / 4 = 10.5\text{ms}$

(2) 见上文

(3) 见上文

Cache 写策略

1. 写命中

- 回写法 (write-back)
 - 当 CPU 对 Cache 写命中时, 只修改 Cache 的内容, 而不立即写入主存, 只有当此块被换出时才写回主存
 - 减少了访存次数, 但存在数据不一致的隐患
- 全写法 (写直通法, write-through)
 - CPU 往 Cache 写数据的同时也往主存中写数据, 一直保存数据一致
 - 由于 CPU 往主存中写数据很慢, 所以需要写缓冲队列 (**写缓冲解决速度不匹配的问题**)
 - 当往 Cache 写数据时, 会按照某种方式往写缓冲中写一份, 然后 CPU 可以继续做其他事情, 写缓冲中的数据由专门的硬件电路写入主存 (**异步, 意味着计算 miss 时间时只需计算写缓冲访问时间, 而无需计算访存时间**)
 - 当写频繁时, 有可能出现写缓冲满的情况, 这时 CPU 会因为写缓冲饱和而发生阻塞

2. 写不命中

- 写分配法 (write-allocate)
 - 未命中, 则先将数据从主存调入 Cache, 然后在 Cache 中进行写, 最后用 **写回法** 将 Cache 中的数据写回主存
- 非写分配法 (not-write-allocate)
 - 当 CPU 对 Cache 写不命中时只写入主存, 不调入 Cache, 只有读未命中时才调入 Cache
 - 搭配 **全写法** 使用

写回 (Write-back) + 写分配 (Write-allocate) 组合的核心思想

1. 既然写操作已经把数据调入 Cache, 那么后续对该块的多次写操作都在 Cache 中完成, 不再访问主存
2. 因此:
 - 写分配: 让未来的写操作可以命中 Cache
 - 写回: 将多个写操作合并成一次“换出时写回主存”
 - 即逻辑链: 写回需要缓存写入的块 → 写分配恰好会把块调入 Cache → 两者自然捆绑使用

为什么全写法搭配非写分配?

全写法每次写都会更新主存, 因此不需要把块调入 Cache; 调入反而浪费 Cache 空间

组合	原因
写回 + 写分配	写回要求所有写都在 Cache 内进行 → 需要写分配将块调入 Cache
写直通 + 非写分配	写直通已更新主存, 不需要把数据调进 Cache

例题

1. 下列关于 Cache 写策略的描述中, 正确的是 (C)
 - A. 写回法在每次写命中时都会更新主存
 - B. 全写法通常与写分配法搭配使用
 - C. 非写分配法在写不命中时不将数据调入 Cache
 - D. 写缓冲主要用于减少写回法的开销
2. 在采用写回法和写分配法的 Cache 系统中, 当发生写不命中时, (B)
 - A. 直接写入主存, 不修改 Cache
 - B. 将数据块调入 Cache 后在 Cache 中修改
 - C. 同时更新 Cache 和主存
 - D. 仅更新写缓冲队列

3. 某计算机系统的 Cache 采用写回法和写分配法，主存访问时间为 100ns，Cache 访问时间为 10ns，写缓冲访问时间为 5ns。假设某程序执行过程中有 1000 次写操作，其中 800 次写命中，200 次写不命中。请计算：

(1) 该系统的平均写访问时间

(2) 如果改为全写法+非写分配法，平均写访问时间是多少？

(3) 从性能角度分析两种策略各自的适用场景

(1) 写回法 + 写分配法的平均写访问时间

写命中 (800 次)：只需写入 Cache，访问时间为 10 ns

写不命中 (200 次)：需要先将数据块从主存调入 Cache (主存访问时间 100 ns)，然后写入 Cache (Cache 访问时间 10 ns)，因此总时间为 100 ns + 10 ns = 110 ns

$$\text{总时间} = 800 \times 10 + 200 \times 110 = 8000 + 22000 = 30000ns$$

$$\text{平均写访问时间} = \frac{30000}{1000} = 30ns$$

(2) 改为全写法 + 非写分配法的平均写访问时间

写命中 (800 次)：需要写入 Cache 和主存，但通过写缓冲优化。假设写缓冲不满，CPU 只需等待 Cache 写入 (10 ns) 和写缓冲写入 (5 ns) 中的 **较慢者**。由于 Cache 访问时间 (10 ns) 大于写缓冲访问时间 (5 ns)，写命中时间取 Cache 访问时间 10 ns (写缓冲异步操作，不增加延迟)

写不命中 (200 次)：直接通过写缓冲写入主存，时间为写缓冲访问时间 5 ns。写缓冲再将数据异步写入主存 (100ns，**但不阻塞 CPU**)。CPU 的等待时间即是写缓冲的访问时间 5ns

$$\text{总时间} = 800 \times 10 + 200 \times 5 = 8000 + 1000 = 9000ns$$

$$\text{平均写访问时间} = \frac{9000}{1000} = 9ns$$

(3)

• 写回法 + 写分配法

- 优点：写命中时只更新 Cache，不立即写主存，减少了主存访问次数；适用于写操作频繁且具有空间局部性的应用（如科学计算、图像处理），能有效降低平均写访问时间
- 缺点：写不命中时开销较大（需调入块并写 Cache），且数据一致性较差（主存更新延迟）；实现较复杂，需维护脏位
- 适用场景：写密集型工作负载，其中写命中率高，且对数据一致性要求不严格的系统

• 全写法 + 非写分配法

- 优点：写操作立即更新主存，数据一致性好；写不命中时直接写主存 via 写缓冲，开销较小；实现简单
- 缺点：写命中时仍需更新主存（尽管写缓冲优化），写缓冲满时可能引起 CPU 阻塞；主存访问频繁，可能成为瓶颈
- 适用场景：写操作较少或需要强一致性的应用（如 I/O 缓冲、多处理器系统），其中写不命中比例较高或写缓冲效率高

三种映射下 Tag 的作用

CPU 需要的主存块数据 ← cache [物理地址]

- tag 的本质，无论哪种映射：用于判断当前 cache 行中的数据是否对应 CPU 所访问的那一块主存数据
- 直接映射：
 - 每个主存块只能映射到唯一的某一 cache 行
 - 多个主存块可能映射到同一行
 - tag 用来区分这些“冲突的主存块”（cache+行号唯一定位 cache 行是否存放目标主存块，block offset 定位块内字节）
- 全相联映射：
 - 任何主存块可以放入 cache 任意位置，tag 唯一标识主存块
- 组相联映射：
 - cache 被划分成多个组
 - 每个主存块映射到某一组

- 但可以放在该组内任一行
- 组号选择组, tag 用于区分该组内的不同主存块

Cache 总容量的计算

$Cache \text{ 容量} = Cache \text{ 行数} \times \text{每一行占的空间}$

每一行占的空间的组成 (把每一部分加起来即可, 题目可能忽略其中部分值)

- 数据 (块大小)
- 标记项 (单位: b)
 - tag
 - 有效位: 1 位
 - 脏位 (修改位、一致性维护位)
 - 全写法: 0 位
 - 回写法: 1 位
 - 替换算法位 (LRU 需要用)
 - 直接映射: 0 位
 - 全相联映射: $\log_2 (Cache \text{ 行数})$
 - 组相联映射: $\log_2 (Cache \text{ 路数})$

主存交叉存储对总线的影响

2013-43

主存采用交叉存储方式, 支持突发传送, 流水线稳定时, 每个总线时钟周期传输一个字 (存储字长)

- 可以用来算总线的带宽 (**最大** 数据传输速率): $\frac{\text{存储字长}}{\text{总线时钟周期}}$
- 若支持突发传送总线事务, 可以用来算事务时间 (传送首地址和命令+存储器准备数据+传送数据)
 - 传送首地址和命令: 一般题目会给, 比如需要一个总线时钟周期
 - 存储器准备数据: 流水线画甘特图计算
 - 传送数据: 一般题目会给

408 中 **流水线技术的应用** 主要集中在以下两个地方

- CPU 指令流水线
- 主存多体交叉存储
- 分组交换的时延

磁盘相关计算

磁盘物理地址: 柱面号、磁头号、扇区号

- 柱面号: 每个盘面的磁道数取 $\log$
- 磁头号: 盘面数取 $\log$
- 扇区号: 每个磁道的扇区数取 $\log$

扇区访问时间: 寻道时间 + 平均延迟时间 + 平均传输时间

- 寻道时间: 一般题目会给平均的
- 平均延迟时间: 磁盘转半圈所用的时间
- 平均传输时间: 一个扇区划过磁头下方所需要的时间 = 磁盘转一圈的时间 / 磁道的扇区数

附录：说明

- 1. **W**：错误
 - A**：模棱两可或看答案才恍然大悟
 - N**：课本上没有或新考法
- 2. **F**：一级错题，做错过或模棱两可或看答案才恍然大悟的题
 - S**：二级错题，**不看答案重做** 一级错题做错或不会的题
- 3. **M**：记忆题